

RAUL LAB · DOCUMENTACIÓN CANÓNICA DERIVADA DE MKDOCS

Replicant Lab

Infraestructura, hosts, red, despliegues y operación.

Versión reproducible

sha256:5a981573e67e4f5a9fde6aee8e0f8ea4a1890e80ec5b139d68418c4400f65c09

Páginas fuente

46

Diagramas Mermaid

10

Índice

[Inicio](#)[Arquitectura](#)[Evolución](#)

HOSTS

[Replicant](#)[Nexus](#)[DigitalOcean](#)[GitHub](#)

RED

[Visión general](#)[Inventario provisional](#)

DESPLIEGUE

[Modelos](#)[Git](#)[Docker](#)

OPERACIÓN

[SSH](#)[Comandos útiles](#)[Mantenimiento](#)[Descargas](#)

PENDIENTES

[Resumen](#)[Cartera Estratégica](#)[PULA](#)[CV / Firebase](#)[Control de Red](#)[Nexus](#)[App Launch](#)

APLICACIONES

[Índice](#)[PULA](#)[App Launch](#)[ErasmusHomes · Control del MVP](#)[Salones AV](#)[Reserva-Pistas-UTP](#)[Consumos Cupra](#)[CV de Raúl](#)[Control de Red](#)[Cartera Estratégica](#)[Replicant Lab](#)

GOBIERNO DEL TRABAJO

[Flujo Work–Codex–Git](#)[Instrucciones para Work](#)[Sistema de encargos](#)[Plantilla de encargo](#)[GOV-001](#)[Autodocumentación](#)[Decisiones](#)

CHANGE LOG

[Índice](#)[21 de agosto de 2026](#)[13 de agosto de 2026](#)[9 de agosto de 2026](#)[8 de agosto de 2026](#)

Replicant Lab

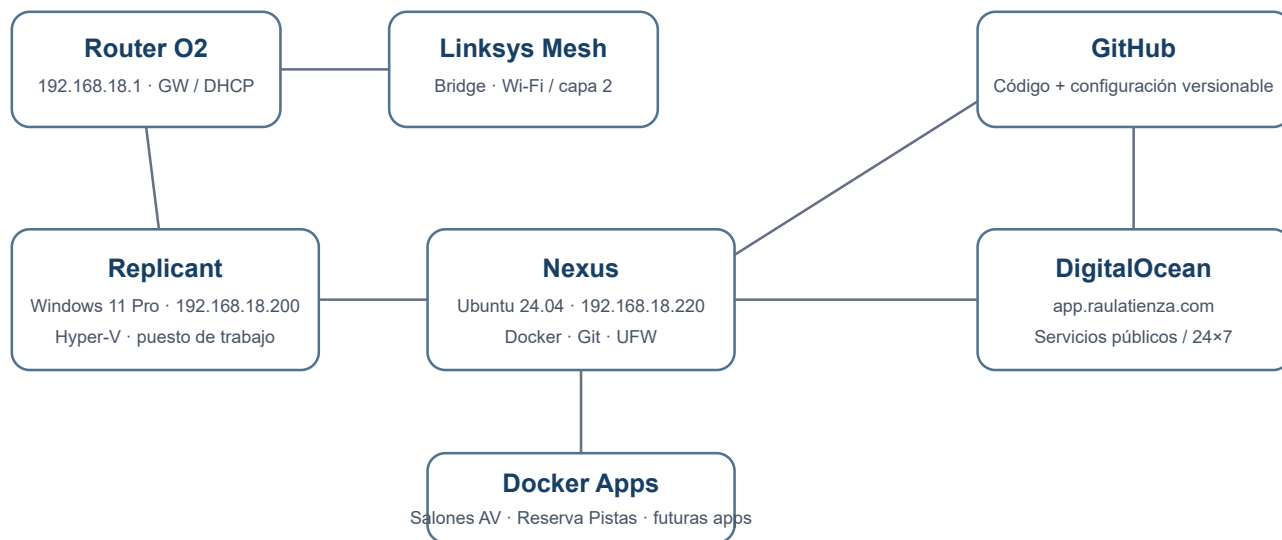
Manual vivo de la infraestructura local y cloud del laboratorio: qué existe, dónde se ejecuta, cómo se opera y qué queda pendiente.

Objetivo

Mantener una visión única, entendible y versionada de **concepto, hosts, red, seguridad, Git, Docker y operación**. La documentación funcional de cada aplicación vive en su propio repositorio.

Las versiones portables se descargan desde el icono ↓ **Descargas** de la cabecera, junto al selector claro/oscuro.

Arquitectura de un vistazo



La portada usa **SVG nativo**, no Mermaid. Así evitamos que una incompatibilidad del parser pueda romper la vista principal.

Principios

- **Minimalismo:** pocas piezas y cada una con una función clara.
- **Git como fuente de verdad:** código y configuración versionable viven en GitHub.
- **Separación:** Windows para trabajo interactivo; Ubuntu para servicios; cloud para disponibilidad pública/24x7.
- **Persistencia fuera de Git:** datos, secretos y backups no se mezclan con repositorios.
- **Seguridad práctica:** SSH por clave, UFW y puertos publicados de forma explícita.
- **Reproducibilidad:** un host debe poder reconstruirse sin depender de cambios manuales no documentados.
- **Separación:** Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local.

Estado actual

Componente	Estado
Replicant	✅ Operativo

Componente	Estado
Nexus	✔ Operativo
SSH por clave	✔ Operativo
UFW	✔ Activo
Docker / Compose	✔ Operativo
GitHub desde Nexus	✔ Operativo
App Launch	✔ Multientorno validado · Nexus 80 + DigitalOcean HTTP/HTTPS
Puerto 8080	✔ Libre y no asignado
Salones AV	✔ Operativa en Nexus · 8081 · Git/Nexus reconciliados en 8c0bc08
Replicant Lab en Nexus	✔ Runtime Nginx estático validado · 8082
Reserva-Pistas-UTP	✔ Validada y observada en Nexus · 8083
Cartera Estratégica	✔ MVP local en Replicant · no desplegada en Nexus
Reserva-Pistas histórico en Nexus	✔ 18 registros reconciliados bajo demanda · sin conflictos
Consumos Cupra	✔ Cloud Run · 9f66a368 · revisión 00009-pon al 100 %
CV	✔ Firebase Hosting · HTTP 200 · deuda POST-CARTERA
Control de Red	✔ Herramienta local · separación de datos POST-CARTERA
Producción DigitalOcean	✔ App Launch y Reservas; no es runtime de Consumos ni CV
Backups Nexus	⌚ POST-CARTERA
Nombres locales	✔ replicant y nexus mediante hosts en Replicant

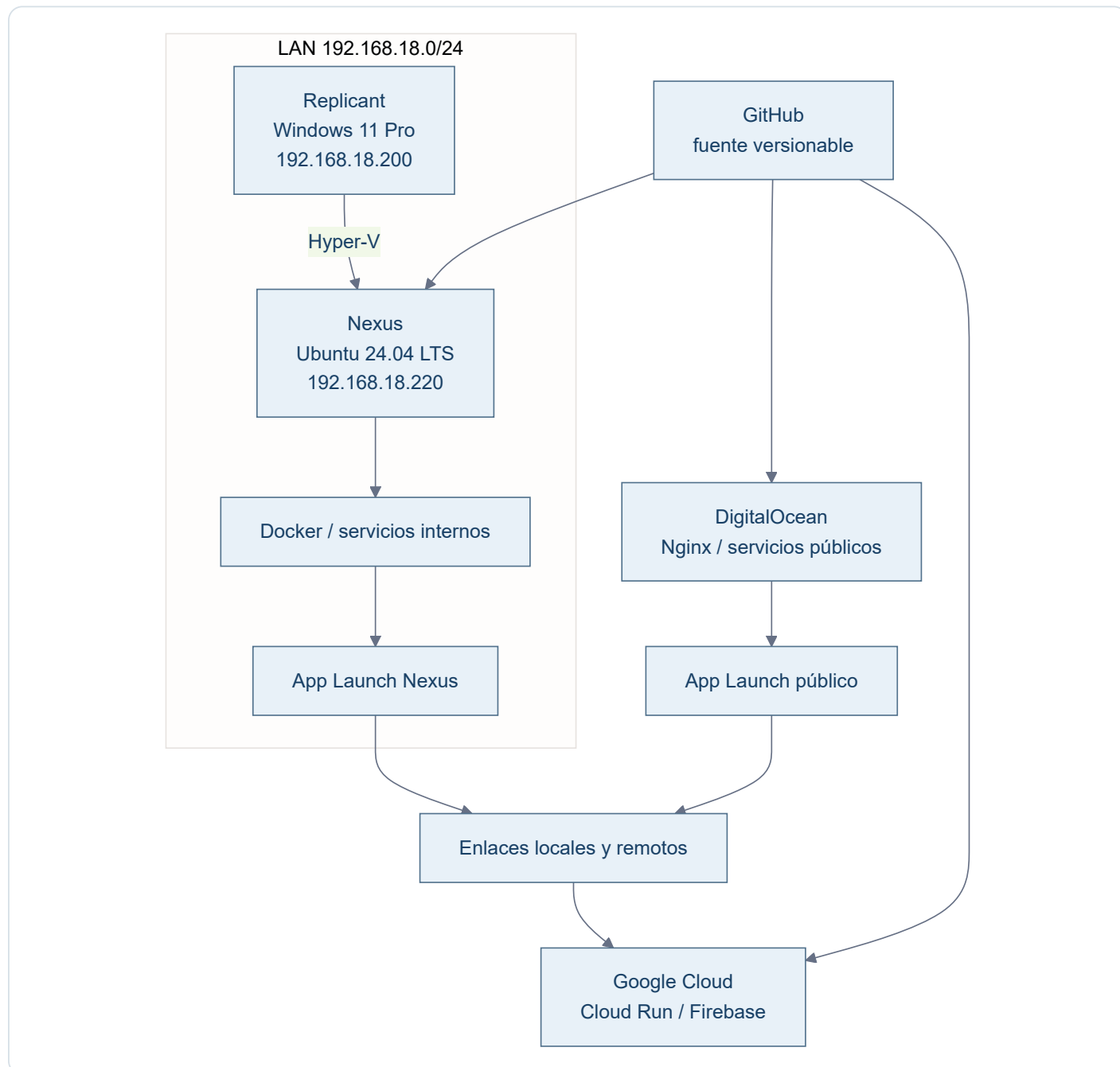
Cómo usar esta documentación

- **Arquitectura** explica cómo encajan las piezas.
- **Evolución** resume hitos y estado actual; el detalle histórico vive en Change Log.
- **Hosts** describe cada máquina.
- **Red** documenta direccionamiento e inventario.
- **Despliegue** explica los modelos heterogéneos: local, Docker, VPS, Cloud Run y Firebase.
- **Aplicaciones** contiene solo la ficha de infraestructura de cada app.
- **Operación** concentra comandos y procedimientos cortos.
- **Pendientes** conserva el estado y los encargos que deben retomarse sin depender de memoria de chat.
- **Decisiones** registra criterios que no conviene redescubrir cada vez.
- **Descargas** ofrece el documento global y las fichas HTML/PDF derivadas de cada aplicación.

Los estados distinguen entre contenido **implementado en Git**, **probado localmente**, **validado u observado en Nexus** y **validado en producción**. Una evidencia de un entorno no se extrapola a otro.

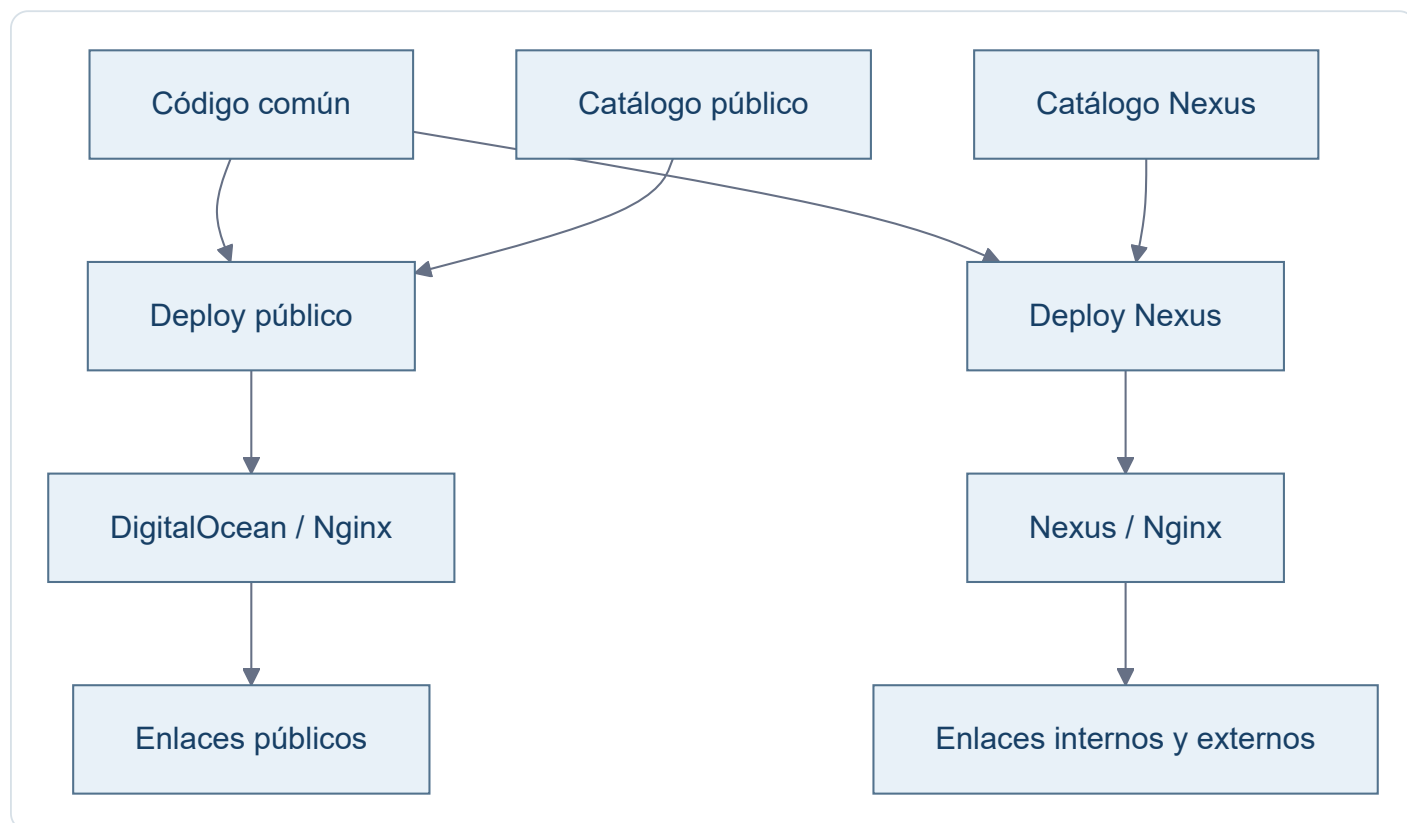
Arquitectura

Modelo general



App Launch es catálogo y capa de acceso. No ejecuta las aplicaciones enlazadas ni demuestra que residan en el mismo host.

App Launch multientorno



El catálogo se selecciona durante el despliegue. Cada host recibe únicamente su `apps.json` ; la lógica visual es común.

Responsabilidades

Elemento	Responsabilidad
Replicant	Estación principal Windows, desarrollo local, Hyper-V y administración
Nexus	Laboratorio Linux, Docker, servicios internos y copias de consulta
GitHub	Código, configuración versionable e histórico
DigitalOcean	App Launch público, Reservas y otros servicios publicados en el VPS
Google Cloud Run	Producción canónica de Consumos Cupra
Firebase Hosting	Producción pública del CV
App Launch	Catálogo y navegación hacia aplicaciones locales o remotas
<code>/opt/data</code> , <code>/opt/secrets</code> , <code>/opt/backups</code>	Datos, secretos y copias fuera de Git

Dos reglas de lectura

Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local .

Docker es el patrón preferido para servicios internos de Nexus cuando encaja, no un requisito universal para todas las aplicaciones.

Evolución del laboratorio

La historia detallada se conserva en el [Change Log](#). Para operar el laboratorio basta con estos hitos:

Etapa	Resultado
Construcción inicial	Replicant quedó como estación principal y host Hyper-V; Nexus como laboratorio Ubuntu/Docker.
Auditoría y normalización	Se inventariaron hosts, red, aplicaciones, checkouts y runtimes sin confundir presencia de código con ejecución.
Remediaciones	Salones AV quedó reconciliado en Git/Nexus; Consumos Cupra quedó trazado y restaurado en Cloud Run; el CV quedó identificado en Firebase Hosting.
Estado actual	Las fases 2A, 2B y 2C están cerradas. No hay una incidencia crítica abierta en esas aplicaciones.
Trabajo futuro	La deuda aceptada se concentra en Pendientes y se retomará después de Cartera Estratégica.

Criterio de validación

Cada aplicación se valida con los mecanismos apropiados para su tecnología: tests, lint, build, configuración, Compose, healthchecks, runtime o HTTP. La evidencia concreta vive en su [ficha técnica](#), sin duplicar aquí los listados de pruebas.

Host · Replicant

Campo	Valor
Host	Replicant
Rol	Estación de trabajo + host Hyper-V
SO	Windows 11 Pro
IP	192.168.18.200
Gateway	192.168.18.1
Virtualización	Hyper-V
Switch	Replicant Ethernet
Acceso	Sesión local de Windows o Escritorio remoto: mstsc /v:replicant

Responsabilidades

- Herramientas interactivas: ChatGPT, Codex, Gemini y similares.
- Administración de Nexus.
- Hyper-V.
- Punto de entrada SSH hacia Nexus y DigitalOcean.

Criterio

No convertir Replicant en servidor por comodidad si el servicio puede vivir limpiamente en Nexus.

Host · Nexus

Campo	Valor
Hostname	nexus
SO	Ubuntu Server 24.04.4 LTS
IP	192.168.18.220/24
Gateway	192.168.18.1
Interfaz	eth0
MAC	00:15:5d:12:7b:00
Plataforma	VM Hyper-V Gen2
Acceso	ssh raul@nexus
Función	Laboratorio Linux, Docker, servicios internos y copias de consulta

Software base

- OpenSSH Server.
- Docker Engine CE.
- Docker Compose.
- Git.
- UFW.
- Nano.
- unattended-upgrades.

Estructura

/opt/apps repositorios y aplicaciones /opt/data datos persistentes /opt/backups copias /opt/compose composiciones separadas si hacen falta
/opt/secrets secretos y .env fuera de Git

Seguridad

Estado

SSH por clave, UFW activo y actualizaciones automáticas de seguridad habilitadas.

Puertos conocidos

Puerto	Uso	Ámbito
22/tcp	SSH	LAN

Puerto	Uso	Ámbito
80/tcp	App Launch	LAN
8080/tcp	Libre	Sin listener
8081/tcp	Salones AV	LAN
8082/tcp	Replicant Lab · Nginx estático	LAN
8083/tcp	Reserva-Pistas-UTP · Nginx	LAN
53	systemd-resolved	localhost

Estado observado el 13/08/2026

- Docker y `unattended-upgrades` activos.
- Replicant Lab ejecutándose como sitio estático en Nginx, construido desde el `Dockerfile` y publicado en `192.168.18.220:8082` → `80`, sin bind mounts ni servidor de desarrollo.
- Salones AV accesible mediante Nginx en `192.168.18.220:8081`; checkout limpio e idéntico a GitHub `main` en `8c0bc08` después de la Fase 2A.
- Reserva-Pistas-UTP ejecutándose como backend privado y proxy Nginx en `192.168.18.220:8083`, con autenticación, datos persistentes separados y canal saliente SSH restringido hacia DigitalOcean.
- App Launch ejecutándose en el puerto `80` mediante Nginx `1.27-alpine`, con sitio y configuración montados en solo lectura.
- `8080` sin listener.
- CV y Control de Red presentes solo como checkouts; no son servicios Nexus.
- Consumos Cupra y Cartera Estratégica sin checkout servido ni puerto Nexus.
- La tarjeta PULA de App Launch apunta a la publicación pública externa y fue validada en el catálogo Nexus `2d265ee`; no implica ejecución de PULA en Nexus.

Esta observación confirma el estado del laboratorio privado en esa fecha. DigitalOcean se validó por separado y de forma acotada para el cierre de Reserva-Pistas-UTP; cada repositorio conserva sus definiciones operativas.

Runtime documental validado en Nexus

El PR #9 fusionado sustituye `mkdocs serve` y los bind mounts por una imagen construida desde el `Dockerfile`: MkDocs estricto en la etapa builder y Nginx estático en runtime, manteniendo `192.168.18.220:8082`. El 09/08/2026 se reconstruyó y recreó exclusivamente este servicio en Nexus y se validaron HTTP, navegación, recursos, cinco diagramas Mermaid y descargas HTML/PDF idénticas byte a byte a los artefactos versionados. El HTML funciona offline sin dependencias esenciales externas y el PDF conserva la documentación completa.

Host · DigitalOcean

Campo	Valor
Alias SSH	docean
Host	app.raulatienza.com
SO	Linux en VPS DigitalOcean
Acceso	Desde Replicant: <code>ssh docean</code> (SSH administrativo por clave)
Función	Nginx, App Launch público y servicios web alojados en el VPS

Modelo operativo

DigitalOcean publica App Launch como capa de navegación y aloja Reserva-Pistas-UTP. Debe mantener despliegues desde fuentes versionadas, secretos fuera de Git y mínima configuración manual.

App Launch puede enlazar servicios externos. En particular, Consumos Cupra se ejecuta en Google Cloud Run, el CV en Firebase Hosting y PULA en su publicación pública de Google/AI Studio; sus tarjetas no los convierten en runtimes de DigitalOcean.

Estado validado

- App Launch respondió `200` por HTTP/HTTPS y sus assets y catálogo público fueron verificados; la tarjeta PULA apunta a la URL pública exacta y figura en el catálogo versionado `2d265ee`.
- Reserva-Pistas-UTP mantiene backend local y publicación mediante Nginx con autenticación.
- El canal de sincronización de Reservas con Nexus usa una clave dedicada y un comando SSH forzado, sin shell, TTY ni forwarding.
- No existe sincronización automática de los datos de Reservas: el operador revisa una vista previa y confirma una dirección.

La documentación no incluye contraseñas, claves privadas ni valores de secretos. Los cambios de DNS, certificados, servicios o datos requieren un encargo específico.

Host lógico · GitHub

GitHub no es un host de ejecución del lab, pero sí una pieza de infraestructura crítica porque actúa como **fuentes de verdad**.

Reglas

- Código y configuración versionable viven aquí.
- Cambios relevantes se realizan en rama.
- Se revisan mediante Pull Request.
- `main` representa el estado estable aprobado.
- Datos, bases de datos, `.env`, tokens y claves privadas no se versionan.

Repos relevantes

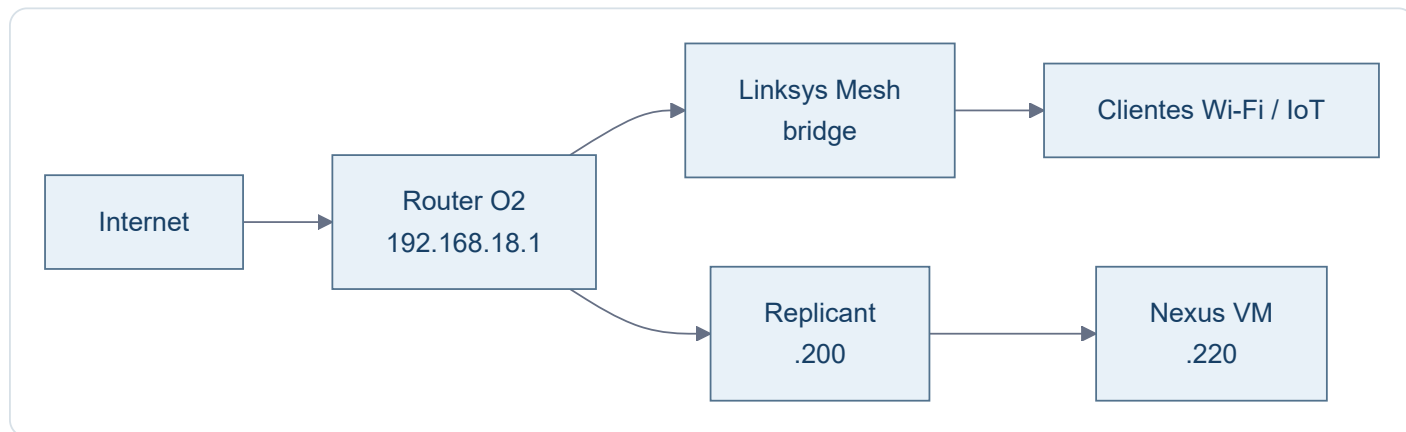
Repo	Función
<code>ratienza/infra-replicant-lab</code>	Documentación viva de infraestructura
<code>ratienza/salones-av-valencia-palace</code>	Aplicación AV / PoC Docker
<code>ratienza/carreira-estrategica</code>	Aplicación de cartera
<code>ratienza/Reserva-Pistas-UTP</code>	Aplicación de reservas de pádel
<code>ratienza/Apps_Launch</code>	Catálogo público e interno de aplicaciones
<code>ratienza/Consumos_Cupra</code>	Aplicación de consumos desplegada en Cloud Run
<code>ratienza/CV-Raul-IA-Estudio-Google-</code>	Fuente del CV publicado en Firebase Hosting
<code>ratienza/control-red</code>	Herramienta local de inventario de red

Cada aplicación conserva en su propio repositorio su código, configuración reproducible y documentación funcional. Este repositorio solo mantiene la visión de infraestructura y las diferencias comprobadas entre entornos.

Un checkout no prueba que exista un runtime, y un trigger conectado a GitHub no se considera producción si la cadena real no está demostrada.

Red · Visión general

Topología



Convención provisional

Rango	Uso previsto
.1	Router / gateway
.2 - .19	Infraestructura de red / mesh
.20 - .179	Clientes, IoT y DHCP general
.180 - .199	Equipos fijos, NAS, impresoras
.200 - .219	Servidores físicos
.220 - .239	VMs / laboratorio
.240 - .254	Reserva futura

DNS

No existe servidor DNS local. Replicant mantiene dos aliases puntuales en el archivo `hosts` de Windows:

Nombre	IP	Uso
replicant	192.168.18.200	Escritorio remoto y herramientas del host
nexus	192.168.18.220	SSH y <code>http://nexus/</code>

El alias solo resuelve el nombre. El protocolo, puerto, usuario y credenciales siguen perteneciendo a cada servicio.

Red · Inventario provisional

Estado

Inventario de trabajo obtenido del escaneo de agosto de 2026. No se considera una CMDB definitiva y algunos nombres/fabricantes pueden requerir validación posterior.

Infraestructura y equipos conocidos

IP	Nombre / función	MAC / fabricante / nota
.1	Router Fibra O2	2C-96-82-69-25-F2 · gateway
.3	RAN-CASA	HP
.4	Sonos 5	5C-AA-FD-0D-54-36 · no visto
.5	Enchufe Emma	4C-11-AE-14-10-6E · IoT
.6	Keres.local	14-91-82-8D-8C-6A · Linksys
.7	SONOS Cocina	B8-E9-37-E7-51-2E
.8	Enchufe ordenador RAN-CASA	D4-A6-51-E5-0F-24 · Tuya
.9	Termostato Nest	18-B4-30-C4-52-48
.10	Sonos Habitación	B8-E9-37-E1-8E-AE
.11	Echo Darío	F4-03-2A-7B-1B-1B
.12	Alexa Comedor	DC-91-BF-D1-35-B3
.13	Linksys13497.local	14-91-82-8D-8C-B2 · Linksys
.14	Alexa Despacho	20-A1-71-00-43-44
.18	Play Darío	0C-FE-45-37-AA-4F · Sony
.19	Alexa Cocina	1C-FE-2B-4F-5A-1B
.20	Fire Stick	34-AF-B3-DE-58-06
.25	IoT desconocido	28-6D-CD-49-AF-28 · no visto
.27	Lamparita Darío	28-6D-CD-56-C8-41
.35	Lamparita comedor	28-6D-CD-5D-BD-81
.42	Chromecast antiguo	8A-05-36-D5-F8-2D · no visto
.44	Escalera	40-F5-20-C1-C7-FB · luces · no visto
.45	Lamparita comedor escalera	28-6D-CD-49-AB-D1
.52	Enchufe Raúl	D4-A6-51-E6-94-8B
.53	Sebastian - CECOTEC	44-87-63-97-02-EA

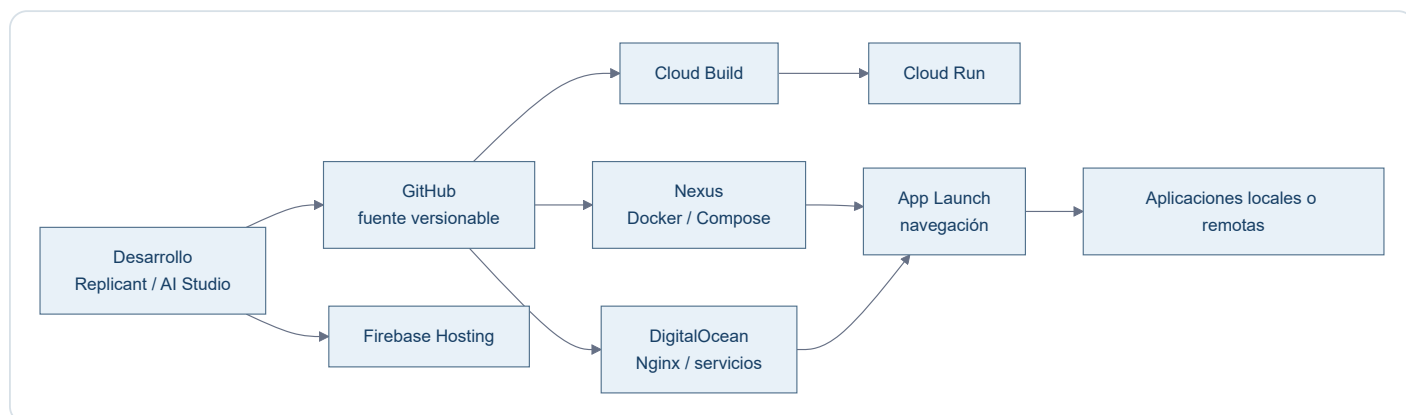
IP	Nombre / función	MAC / fabricante / nota
.54	Impresora Darío	28-C5-C8-AB-D7-BE · HP
.71	Luz despacho	CE-CC-5A-B6-E9-BC · no visto
.74	Móvil Emma	76-9D-67-FE-0D-CA · Samsung · no visto
.82	Chromecast	F6-45-CE-C2-7C-F6
.83	Móvil Darío	BA-D8-D1-F2-04-44 · no visto
.94	Portátil SH	28-92-00-45-01-CD · HP · no visto
.101	Móvil Emma antiguo	58-02-05-B8-4E-BA · no visto
.106	Móvil Raúl	52-1E-45-1D-FB-AA
.123	Antigua IP DHCP de Replicant	00-E0-4C-4B-35-AD · migrado a .200
.124	No identificado	A4-E8-8D-08-12-44 · no visto
.125	Antigua IP DHCP de Nexus	00-15-5D-12-7B-00 · migrado a .220
.200	Replicant	host físico Windows / Hyper-V · IP fija
.220	Nexus	VM Ubuntu / Docker · IP fija

Nota operativa

El inventario sirve para orientación y futuras decisiones de direccionamiento. No se reubicarán dispositivos por estética ni se convertirán en IP fija salvo necesidad concreta.

Modelos de despliegue

Replicant Lab es deliberadamente heterogéneo: el runtime se elige por las necesidades de cada aplicación y no por una regla única.



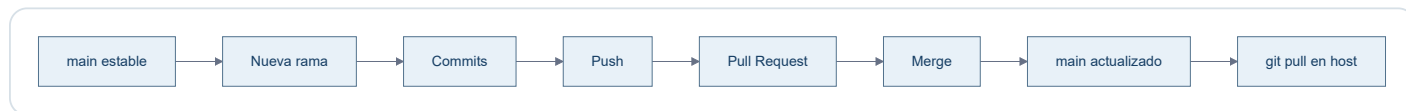
Aplicación	Creación / desarrollo	Deploy	Runtime
Salones AV	Codex / GitHub	Docker Compose	Nexus
Reserva-Pistas-UTP	Codex / GitHub	Compose en Nexus y despliegue controlado en VPS	Nexus + DigitalOcean
Consumos Cupra	AI Studio + Codex / GitHub	Cloud Build + Artifact Registry	Google Cloud Run
CV	AI Studio / GitHub	Despliegue manual observado	Firebase Hosting
Cartera Estratégica	Codex / desarrollo local	Ejecución local	Replicant
Control de Red	PowerShell + Codex	Ejecución local	Replicant
App Launch	Codex / GitHub	Scripts por destino	Nginx en Nexus y DigitalOcean
Replicant Lab	Codex / GitHub	Docker Compose	Nexus

Principios

- Checkout ≠ Runtime : un repositorio presente en un host puede ser solo una copia de consulta.
- Tarjeta App Launch ≠ Runtime local : una tarjeta es un enlace y no demuestra dónde se ejecuta la aplicación.
- App Launch es una capa de navegación; la autenticación, los datos y el ciclo de vida pertenecen a cada aplicación.
- Docker es el patrón principal en Nexus, no el único modelo del laboratorio.

Despliegue · Git

Flujo normal



Convención de ramas para esta documentación

Las ramas representan **cambios lógicos**, no archivos individuales.

Ejemplos:

- docs/bootstrap-infra
- docs/app-salones
- docs/app-cartera
- docs/network
- docs/backups

Regla

Main

main debe representar siempre la documentación aprobada y coherente con el estado real.

Despliegue · Docker

Este patrón describe los servicios Docker de Nexus. No se extrapola a Cloud Run, Firebase Hosting, servicios del VPS ni aplicaciones locales; la comparación completa está en [Modelos de despliegue](#).

Patrón

```
GitHub ↓ /opt/apps/<repo> ↓ compose.yml ↓ docker compose up -d ↓ servicios
```

Buenas prácticas del lab

- Preferir imágenes oficiales cuando sea razonable.
- Minimizar paquetes instalados en el host.
- Persistencia fuera del contenedor.
- Secretos fuera de Git.
- Publicar solo puertos necesarios.
- Cuando proceda, ligar puertos a `192.168.18.220` en vez de `0.0.0.0`.
- Separar proxy, backend y datos cuando cada pieza tenga una responsabilidad clara.
- Ejecutar los procesos de aplicación con un usuario no root siempre que sea viable.
- Usar `restart: unless-stopped` para servicios que deben recuperarse tras reiniciar Nexus.
- No añadir paneles de administración si no resuelven un problema real.

Convención de puertos en Nexus

Puerto	Servicio	Estado
80	App Launch Nexus	Operativo
8080	Sin asignar	Libre
8081	Salones AV	Operativo
8082	Replicant Lab · documentación	Operativo
8083	Reserva-Pistas-UTP	Operativo
8084+	Próximos servicios	Asignación secuencial

Reglas:

- App Launch usa el puerto estándar `80`, sin puerto explícito en la URL.
- `8080` queda libre y no se reserva para App Launch.
- A partir de `8081`, los servicios se numeran consecutivamente salvo necesidad técnica justificada.
- Cada nuevo servicio debe quedar documentado aquí al asignar su puerto.
- Siempre que sea posible, publicar el servicio ligado a `192.168.18.220` y no a `0.0.0.0`.

App Launch es la excepción deliberada: publica `80` en todas las interfaces de Nexus para actuar como entrada de la LAN. No se publica por HTTPS.

Salones AV aplica explícitamente la regla LAN mediante `192.168.18.220:8081:80`. El PR `salones-av-valencia-palace#2` incorporó el bind a `main`; desde `8c0bc08` el checkout Nexus está limpio y la configuración es reproducible desde GitHub.

Reserva-Pistas-UTP · patrón validado

Reserva-Pistas utiliza un Compose de dos servicios:

```
LAN :8083 ↓ nginx ↓ red privada Compose ↓ app:8765
```

El backend no publica `8765` hacia el host. Nginx lo alcanza mediante el DNS interno de Docker usando el nombre de servicio `app`.

Los datos se desacoplan del ciclo de vida del contenedor mediante:

```
/opt/data/reserva-pistas:/app/data
```

La aplicación recibe por entorno:

```
APP_HOST=0.0.0.0 APP_PORT=8765 APP_DATA_DIR=/app/data
```

El código conserva valores por defecto compatibles con despliegues no Docker.

Operación estándar

```
cd /opt/apps/<proyecto> git switch main git pull docker compose up -d --build
```

Comprobaciones habituales:

```
docker compose ps docker compose logs --tail 50
```

Parada y recreación:

```
docker compose down docker compose up -d
```

Un `down` elimina contenedores y la red del proyecto, pero no debe eliminar datos persistentes alojados fuera del contenedor.

Autoarranque

Docker Engine arranca con Ubuntu. Los contenedores existentes con `restart: unless-stopped` se recuperan automáticamente después de reiniciar Nexus.

Compose no necesita ejecutarse manualmente durante el arranque para recuperar esos contenedores ya creados; su fichero YAML sigue siendo la definición reproducible para recrearlos o actualizarlos.

Replicant Lab · sitio estático reproducible

La definición versionada converge en un único modelo:

```
mkdocs.yml + docs/ ↓ Dockerfile · MkDocs build --strict ↓ sitio estático ↓ Nginx :80 ↓ 192.168.18.220:8082
```

`compose.yml` construye el `Dockerfile`, publica únicamente `192.168.18.220:8082 → 80` y no monta fuentes ni ejecuta `mkdocs serve`.

Actualización desplegada

```
cd /opt/apps/infra-replicant-lab git switch main git pull --ff-only origin main docker compose up -d --build
```

Cada cambio documental desplegado requiere reconstruir la imagen porque Nginx sirve la copia estática incluida durante el build. Las dependencias viven en las etapas versionadas; no se instalan en Nexus.

Separación de estados

El modelo estático está implementado, probado localmente y validado en Nexus. La validación del 09/08/2026 incluyó el contenedor Nginx estable, HTTP y navegación, cinco diagramas Mermaid, descargas reales idénticas a Git, HTML offline y PDF completo. No implica validación en producción.

Operación · SSH

Desde Replicant

```
ssh raul@nexus ssh docean
```

`ssh raul@nexus` es la referencia operativa principal. El alias corto puede depender de la configuración SSH local y no debe darse por supuesto.

Configuración conceptual

```
Host nexus HostName 192.168.18.220 User raul IdentityFile C:\Users\raul\.ssh\nexus_local IdentitiesOnly yes Host docean HostName app.raulatienza.com User root IdentityFile C:\Users\raul\.ssh\reserva_pistas_do IdentitiesOnly yes
```

Claves

- `nexus_local` : Replicant → Nexus.
- `reserva_pistas_do` : Replicant → DigitalOcean.
- `~/.ssh/id_ed25519` en Nexus: Nexus → GitHub.

Nunca documentar

No almacenar en este repo claves privadas, tokens, contraseñas ni el contenido de secretos.

Operación · Comandos útiles

Host

```
ip addr ip route sudo ss -tulpn
```

Firewall

```
sudo ufw status verbose
```

Docker

```
docker ps docker compose up -d docker compose down docker compose logs
```

Git

```
git status git switch main git pull
```

Actualizaciones

```
sudo apt update apt list --upgradable sudo apt upgrade
```

Filosofía

Este documento evita convertirse en un recetario infinito. Solo se incluyen comandos frecuentes y útiles para operar el lab.

Operación · Mantenimiento

Actualizaciones

`unattended-upgrades` está activo y habilitado. Ubuntu puede diferir ciertos paquetes mediante phased rollout; no se fuerzan salvo necesidad.

Comprobación:

```
systemctl status unattended-upgrades --no-pager
```

Limpieza

`apt autoremove` puede utilizarse tras revisar qué paquetes se eliminarán.

Criterio de mantenimiento

- Actualizar con regularidad.
- No instalar herramientas "por si acaso".
- Revisar servicios expuestos con `ss -tulpn`.
- Mantener `main` coherente con la realidad.

Descargas

La documentación MkDocs de este proyecto es la única referencia canónica. Los dos ficheros siguientes se generan exclusivamente desde `mkdocs.yml`, las páginas de `docs/` incluidas en `nav` y sus recursos versionados.

[↓ Descargar documentación HTML offline](#)[↓ Descargar documentación PDF](#)

Rutas canónicas únicas

Artefacto	Ruta	Cobertura
HTML autocontenido	<code>docs/downloads/Replicant-Lab.html</code>	Toda la navegación MkDocs, índice interno y siete Mermaid
PDF A4	<code>docs/downloads/Replicant-Lab.pdf</code>	Portada, índice, documentación completa, numeración y siete Mermaid

No se mantiene ninguna copia alternativa.

Fichas técnicas individuales

Cada pareja HTML/PDF se genera desde la misma página Markdown incluida en la navegación. El HTML funciona offline y el PDF está preparado para consulta o archivo.

Aplicación	HTML	PDF
PULA	Descargar	Descargar
App Launch	Descargar	Descargar
Salones AV	Descargar	Descargar
Reserva-Pistas-UTP	Descargar	Descargar
Consumos Cupra	Descargar	Descargar
CV de Raúl	Descargar	Descargar
Control de Red	Descargar	Descargar
Cartera Estratégica	Descargar	Descargar
Replicant Lab	Descargar	Descargar

Propiedades verificables

- Todos los artefactos muestran la misma huella SHA-256 de las fuentes.
- El HTML incorpora CSS, JavaScript y Mermaid localmente y funciona mediante `file://` sin red.
- El HTML no solicita CDN ni contiene clientes de recarga, conexiones dinámicas o referencias al servidor de desarrollo.
- El PDF mantiene texto seleccionable, enlaces, tablas, código, avisos y diagramas.
- El JavaScript del sitio calcula las rutas desde su propio recurso y funciona con el sitio estático en `/`.

Regeneración y sincronía

Después de preparar las dependencias fijadas, un único comando regenera y valida todo:

```
python scripts/docs_pipeline.py generate
```

El ejecuta el modo `check`, vuelve a renderizar un PDF de control, verifica la huella y cobertura, construye la imagen Docker final, arranca Nginx temporalmente y compara byte a byte las descargas servidas con los artefactos versionados.

Estado de despliegue

El pipeline y el runtime estático están implementados y probados localmente. El 21/08/2026 se comprobó el dossier global y las nueve parejas de fichas individuales, incluida PULA; los ficheros servidos por el entorno de validación coincidieron byte a byte con los artefactos generados. La publicación en Nexus se valida de nuevo después de integrar el cambio en `main`; esta evidencia corresponde al laboratorio privado y no se extrapola a producción.

Pendientes · Resumen

No hay incidencias críticas abiertas en Salones AV, Consumos Cupra o Reserva-Pistas-UTP. Las fases 2A, 2B y 2C permanecen cerradas.

Elemento	Tipo	Prioridad	Estado
Cartera Estratégica	Bloque activo	Alta	En curso
PULA	Seguridad, operación y calidad	Alta	Pendiente
CV / Firebase	CI, trazabilidad, seguridad y UX	Media	POST-CARTERA
Control de Red	Separación de datos operativos	Media	POST-CARTERA
Nexus	Backups y gobierno de checkouts	Media	POST-CARTERA
App Launch	Limpieza de assets históricos	Baja	Mejora opcional

Cada detalle se mantiene una sola vez en su página de este grupo.

Pendientes · Cartera Estratégica

Bloque activo

Continuar Cartera Estratégica como siguiente bloque de producto.

Criterio operativo

El trabajo, las decisiones y la validación pertenecen al repositorio de Cartera Estratégica. Replicant Lab solo registra que es el bloque activo.

Pendientes · PULA

Seguridad

- Autenticar y autorizar la API antes de cualquier exposición adicional.
- Bloquear SSRF y sacar contactos y estado vivo de Git.

Operación

- Antes de trasladar a DigitalOcean, definir persistencia, imagen y despliegue reproducibles, secretos, healthcheck, backup/restore y rollback.

Calidad

- Corregir el falso estado `sent`.
- Añadir tests aislados y CI.
- Fijar runtime y gestor de dependencias.
- Reconciliar `AGENTS.md` con las funciones presentes.

Mejora

Normalizar extracción, traducción y capacidad de anuncios SCPU y añadir observabilidad sin datos personales.

Pendientes · CV / Firebase

Trazabilidad

- Reconciliar Firebase y Cloud Build.
- Retirar o corregir el trigger y la configuración obsoletos.
- Demostrar la relación entre SHA Git y versión Firebase.

CI y reproducibilidad

- Incorporar lockfile, build reproducible, CI, tests y typecheck.
- Revisar restos React, metadata Gemini y artefactos residuales.

Seguridad

Revisar cabeceras, caché y protección de `main`.

UX

Revisar el revelado de contacto.

Pendientes · Control de Red

Protección previa

Hacer backup antes de intervenir.

Separación de datos

- Separar inventarios, snapshots y datos operativos reales del código Git.
- Mantener únicamente ejemplos anonimizados en el repositorio.

Pendientes · Nexus

Backups

Definir una política global de backups y ejecutar una prueba de restauración.

Checkouts

- Inventariar y gestionar los checkouts que son solo copias de consulta.
- Mantener explícito `Checkout ≠ Runtime`; el CV en Nexus no es un despliegue.

Pendientes · App Launch

Mejora opcional

Limpiar assets PWA históricos no utilizados mediante un cambio versionado y verificable.

Esta mejora no reabre las fases ya cerradas ni representa un fallo de producción.

Aplicaciones

Catálogo operativo del laboratorio. Cada ficha técnica se abre como documento HTML independiente y autocontenido.

PULA

BÚSQUEDA DE ALOJAMIENTO ESTUDIANTEL

POC privada para localizar y gestionar apartamentos publicados por SCPU en Pula, Croacia. Incorpora scraping, importación asistida y seguimiento Gmail; no incluye el concepto independiente ErasmusHomes.

Herramienta	AI Studio + AntigraVity
Stack	React, Vite, Express, Gemini
Repositorio	ratienza/Pula
Deploy	Producción pública existente
Runtime	Google · no modificado
Estado	POC beta Pula / publicada en ambos App Launch
URL	Producción

[Ficha técnica](#) · [Producción](#) · [Documentación](#)

App Launch

PORTAL MULTIENTORNO

Catálogo de acceso a aplicaciones públicas e internas. Se publica en Nexus y DigitalOcean con catálogos distintos por entorno; las tarjetas navegan hacia servicios, pero no constituyen su runtime.

Herramienta	Codex
Stack	HTML, CSS, JavaScript
Repositorio	ratienza/Apps_Launh
Deploy	Scripts por destino
Runtime	Nginx · Nexus y DigitalOcean
Estado	Operativo
URL	Nexus

[Ficha técnica](#) · [Nexus](#) · [Producción](#)

ErasmusHomes · Control del MVP

CONSOLA INTERNA DE DIRECCIÓN

Panel derivado del roadmap canónico de ErasmusHomes. Resume progreso, prioridades, gates, validación de fin de semana, riesgos, evidencias y sincronización con el SHA de GitHub sin convertirse en fuente paralela.

Herramienta	Codex
Stack	MkDocs + YAML generado
Repositorio fuente	ratienza/ErasmusHomes
Repositorio visual	ratienza/infra-replicant-lab
Deploy	Documentación existente
Runtime	Nexus · 8082
Estado	Interno / EH-002
URL	Panel interno

[Ficha técnica](#) · [Panel de control](#)

Salones AV

GUÍA OPERATIVA

Guía audiovisual para los salones del Valencia Palace. Reúne procedimientos, conexiones, sonido y mapas de apoyo para la operación diaria.

Herramienta	Codex
Stack	HTML, Nginx
Repositorio	ratienza/salones-av-valencia-palace
Deploy	Docker Compose
Runtime	Nexus · 8081
Estado	Cerrado / operativo
URL	Nexus

[Ficha técnica](#) · [Nexus](#)

Reserva-Pistas-UTP

RESERVAS Y AUTOMATIZACIÓN

Gestiona reservas de pistas de pádel y su programación. Ofrece interfaz web y bot de Telegram para consultar, crear o cancelar operaciones autorizadas.

Herramienta	Codex
Stack	Python, Flask, Nginx
Repositorio	ratienza/Reserva-Pistas-UTP
Deploy	Compose + systemd/Nginx
Runtime	Nexus y DigitalOcean
Estado	Operativo
URL	Producción

[Ficha técnica](#) · [Producción](#) · [Nexus](#)

Consumos Cupra

REGISTRO Y ANÁLISIS

Registra y analiza consumos del vehículo, histórico, pendientes y estadísticas. Mantiene la información operativa desde una interfaz web conectada con servicios Google.

Herramienta	AI Studio + Codex
Stack	React, Vite, Express
Repositorio	ratienza/Consumos_Cupra
Deploy	Cloud Build
Runtime	Google Cloud Run
Estado	Cerrado / operativo

[Ficha técnica](#) · [Producción](#)

CV de Raúl

PERFIL PROFESIONAL

Presenta el currículum y portfolio profesional público. Organiza experiencia, formación y contacto, con una versión descargable en PDF.

Herramienta	AI Studio
Stack	Vite, Tailwind, PDFKit
Repositorio	ratienza/CV-Raul-IA-Estudio-Google-
Deploy	Firebase Hosting
Runtime	Firebase
Estado	Operativo / POST-CARTERA
URL	Producción

[Ficha técnica](#) · [Producción](#)

Control de Red

INVENTARIO LOCAL

Descubre e inventaría dispositivos de la red local. Ayuda a revisar direccionamiento, nombres y estado observable desde Replicant.

Herramienta	PowerShell + Codex
Stack	PowerShell
Repositorio	ratienza/control-red
Deploy	Ejecución local
Runtime	Replicant
Estado	Operativo local / POST-CARTERA

[Ficha técnica](#) · [Runtime local en Replicant](#) · [sin URL publicada](#)

Cartera Estratégica

ANÁLISIS FINANCIERO

Gestiona y analiza una cartera personal de inversión. Centraliza posiciones, histórico y métricas de seguimiento en una interfaz local.

Herramienta	Codex
Stack	Python, Streamlit, SQLite
Repositorio	ratienza/cartera-estrategica
Deploy	Ejecución local
Runtime	Replicant
Estado	MVP operativo

[Ficha técnica](#) · [Runtime local en Replicant](#) · [sin URL publicada](#)

Replicant Lab

DOCUMENTACIÓN OPERATIVA

Documenta infraestructura, despliegue y operación del laboratorio. Genera un sitio navegable y versiones portables HTML/PDF desde fuentes Markdown canónicas.

Herramienta	Codex
Stack	MkDocs, Mermaid, Nginx
Repositorio	ratienza/infra-replicant-lab
Deploy	Docker Compose
Runtime	Nexus · 8082
Estado	Operativo
URL	Nexus

[Ficha técnica](#) · [Nexus](#)

Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local.

PULA

Auditoría técnica y manual funcional de `ratienza/Pula` . **PULA es exclusivamente una POC privada para localizar y gestionar apartamentos de la web oficial SCPU en Pula, Croacia.** Su finalidad actual es ayudar en una búsqueda familiar concreta; no es ErasmusHomes ni una plataforma paneuropea.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / producción:** [AI Studio](#)
- **Panel de control:** no existe uno independiente documentado.

Alcance de este documento

PULA es una POC privada de búsqueda de apartamentos SCPU en Pula, Croacia. No forma parte de ErasmusHomes.

Estado comprobado

Elemento	Evidencia
Repositorio	ratienza/Pula · privado
main auditado	ff8165cf7c9d1d4c5ae670c56486c25900a5754b
Alcance funcional	Búsqueda SCPU, gestión de candidatos, contacto y seguimiento Gmail
Checkout Nexus	<code>/opt/apps/Pula</code> · limpio y sincronizado; copia de consulta, no runtime
Producción pública	https://pula-erasmus-housing-automator.ai.studio/ · HTTP 200 validado el 21/08/2026; el runtime no se modificó
App Launch	Tarjeta publicada y validada en Nexus y DigitalOcean desde <code>ratienza/Apps_Lauch</code> (2d265ee)
Pruebas locales	TypeScript y build Vite/esbuild correctos sobre copia temporal
Tests automatizados	No existe suite; solo scripts exploratorios de scraping/Firebase
Datos versionados	<code>data.json</code> : 29 registros new con 29 direcciones de contacto

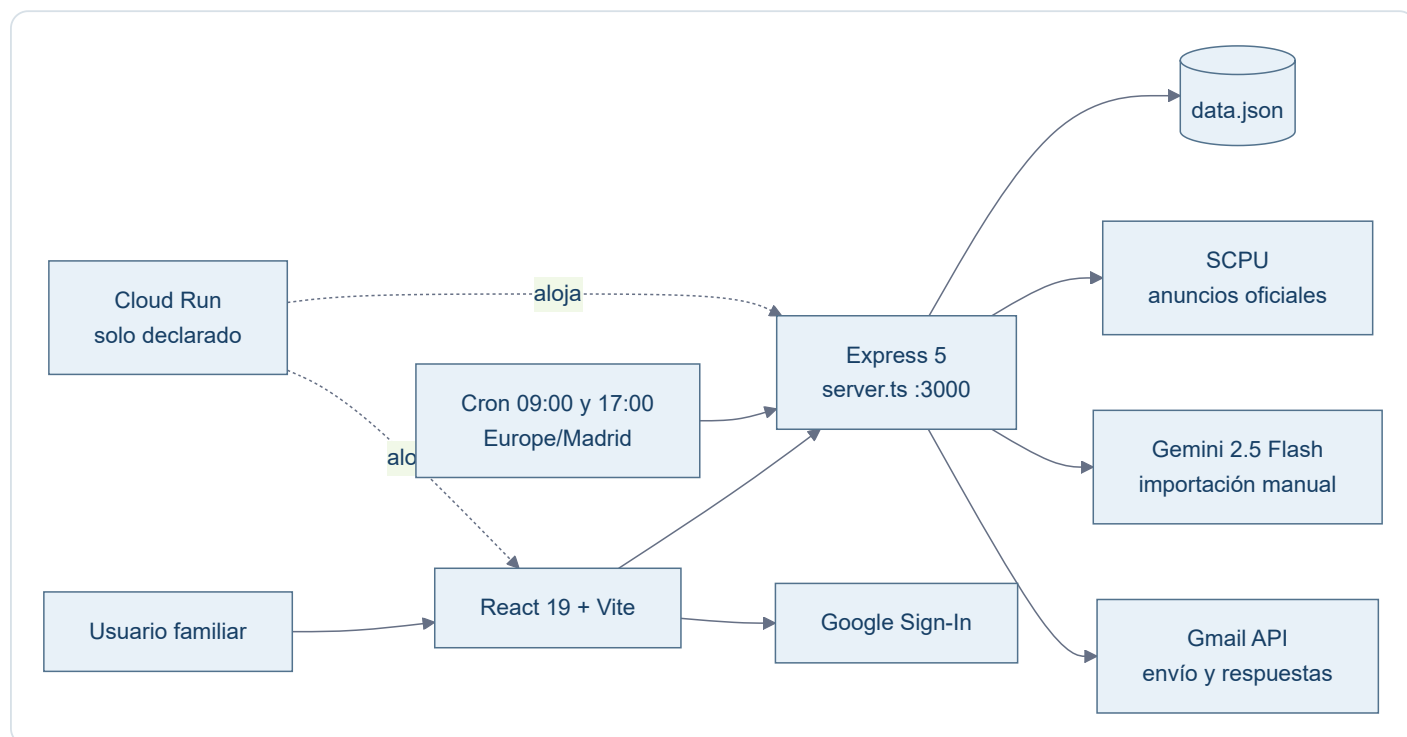
Calidad general: **5/10**. La POC compila y concentra el flujo funcional para Pula, pero carece de tests, control de acceso, persistencia robusta y trazabilidad reproducible de Cloud Run.

Separación obligatoria de proyectos

[ratienza/ErasmusHomes](#) es otro repositorio y otro proyecto. Contiene el concepto comercial paneuropeo, el ejemplo visual de Bolonia/UNIBO, Smart Deposit Audit y el dossier ReportLab. Actualmente debe tratarse como **concepto no validado**, no como funcionalidad implementada de PULA.

La etiqueta `v0.1-poc-beta` y los artefactos ErasmusHomes presentes en el historial de `ratienza/Pula` son contaminación histórica y deuda de higiene del repositorio. No definen el alcance de PULA y no se usan como fuente funcional en esta ficha. Cualquier limpieza futura de esa historia o etiqueta requiere un encargo específico sobre el repositorio PULA.

Arquitectura de `main`



El build ejecuta `vite build` y empaqueta `server.ts` con esbuild como `dist/server.cjs`. El repositorio no contiene Dockerfile, manifiesto Cloud Run, workflow CI ni definición de volumen persistente; Git no permite reproducir ni vincular el despliegue declarado con el SHA auditado.

Modelo de datos

Cada apartamento registra UUID, casero, superficie, capacidad, precio, ubicación, descripción, email, enlace, condiciones, contrato, estado y fecha. Puede añadir origen manual y datos de respuesta. El enlace funciona como clave de deduplicación.

Estados: `new`, `sent`, `following` y `discarded`.

Manual funcional de `main`

Descubrimiento automático

1. **Extraer Nuevos Pisos SCPU** o el cron inicia el proceso.
2. El servidor recorre hasta cinco páginas y limita el lote a 40 enlaces.
3. Cheerio extrae casero, email, precio, superficie, zona y contrato.
4. Se traducen expresiones croatas mediante sustituciones locales.
5. Se excluyen contratos detectados como anuales y se deduplica por URL.
6. El resultado se guarda y aparece en **Nuevas Oportunidades**.

La implementación contradice la afirmación documental "sin límite artificial": hay un máximo de cinco páginas y 40 enlaces. Además, el scraper fija la capacidad como `Ver descripción`, aunque la regla de interfaz exige un número de personas.

Importación manual con IA

1. **Add URL** 🖱️ abre el formulario.
2. El backend descarga la URL y elimina etiquetas ruidosas.
3. Envía hasta 12.000 caracteres a Gemini 2.5 Flash con esquema JSON.
4. Si Gemini no está disponible, aplica una heurística.
5. Marca la ficha como manual y deduplica por URL exacta.

No es segura para exposición pública hasta restringir protocolos, DNS/IP, redes privadas, redirecciones, tamaño y tiempo: hoy una petición anónima puede ordenar al servidor descargar una URL arbitraria, creando riesgo de SSRF.

Contacto y seguimiento Gmail

1. Firebase Google Sign-In entrega un access token conservado en memoria.
2. El backend construye un correo bilingüe inglés/croata y llama a Gmail API.
3. La comprobación revisa hasta 25 hilos y compara remitentes con caseros.
4. Una respuesta mueve un anuncio enviado a `following`.

Existe un defecto de integridad: el endpoint responde `success` y cambia el estado a `sent` incluso si falta un token utilizable o Gmail devuelve un error. “Enviado” no prueba que el correo haya salido.

Gestión visual

La SPA ofrece Nuevas Oportunidades, En Seguimiento, Enviados sin respuesta e Histórico, orden por fecha o precio y cambio manual de estado. Los botones de producto están presentes. `package.json` usa React 19, aunque la arquitectura escrita declara React 18.

Material ErasmusHomes excluido

El dossier de Bolonia, los mockups UNIBO, el backend FastAPI de auditoría de fianzas y los generadores ReportLab se excluyen expresamente de la arquitectura, API, puntuación y manual funcional de PULA. Solo se comprobó que su ubicación canónica actual es el repositorio independiente `ratienza/ErasmusHomes`; no se auditó su funcionamiento.

API REST observada

Método y ruta	Entrada	Salida y efecto	Autenticación observada	Riesgo principal
GET <code>/api/config</code>	Ninguna	JSON con <code>clientId</code> OAuth	No	Expone configuración cliente; no es un secreto por sí misma
GET <code>/api/apartments</code>	Ninguna	Devuelve todos los registros	No	Expone contactos y datos operativos sin autorización
POST <code>/api/apartments/scrape</code>	Ninguna	Consulta SCPU, deduplica y reescribe <code>data.json</code>	No	Escritura anónima, sin timeout ni control de concurrencia
POST <code>/api/apartments/manual-url</code>	JSON <code>{url}</code>	Descarga, analiza y añade un candidato	No	SSRF, consumo abusivo y persistencia anónima
POST <code>/api/gmail/check-replies</code>	Bearer OAuth de Google	Lee hasta 25 hilos y mueve respuestas de <code>sent</code> a <code>following</code>	Bearer requerido	Token delegado y coincidencias heurísticas de remitente
PUT <code>/api/apartments/{id}/status</code>	JSON <code>{status}</code>	Cambia el estado y reescribe el fichero	No	Mutación anónima y posibles escrituras perdidas
POST <code>/api/apartments/{id}/send</code>	Bearer opcional; email alternativo opcional	Intenta enviar correo y cambia a <code>sent</code>	Opcional en la práctica	Puede declarar éxito aunque Gmail no haya enviado

No se observan OpenAPI versionado, rate limiting, sesión de aplicación, autorización por usuario ni validación estructural centralizada.

Datos y persistencia

`data.json` es simultáneamente semilla versionada y base de datos de ejecución. El SHA auditado contiene 29 apartamentos en estado `new` y 29 direcciones de contacto. Cada registro puede incluir identidad del casero, email, enlace, precio, ubicación, descripción, condiciones, contrato, estado, origen manual y metadatos de respuesta.

- **Persistencia implementada:** lectura y escritura síncronas del JSON completo mediante `fs.readFileSync` y `fs.writeFileSync`.
- **Concurrencia:** no hay bloqueo, control de versión ni reemplazo atómico; dos peticiones pueden sobrescribirse.
- **Cloud Run:** el filesystem del contenedor no constituye almacenamiento duradero; Git no define volumen ni servicio externo.
- **Privacidad:** los emails son datos personales y no deberían permanecer en Git ni exponerse por una API anónima.
- **Recuperación:** no existe procedimiento versionado de backup, restauración o migración.

Seguridad

Severidad	Hallazgo confirmado	Evidencia	Recomendación futura
Crítica	API de lectura y mutación sin control de acceso	CORS global y rutas de apartamentos sin middleware de autenticación	Autenticación de aplicación y autorización por usuario
Alta	SSRF en importación manual	El servidor acepta una URL que empiece por <code>http</code> y ejecuta <code>fetch(url)</code>	Lista de protocolos, resolución DNS/IP segura, bloqueo de redes privadas, redirecciones y límites
Alta	Estado <code>sent</code> falso	El bloque normal y el <code>catch</code> persisten <code>sent</code> incluso sin envío confirmado	Cambiar el estado solo tras respuesta válida de Gmail y registrar el error
Alta	Datos personales versionados y expuestos	<code>data.json</code> y <code>GET /api/apartments</code>	Almacén privado, minimización, control de acceso y retirada del histórico Git mediante encargo específico
Media	Escritura no atómica	Se reescribe el JSON completo sin bloqueo	Base de datos persistente o reemplazo atómico con control de concurrencia
Media	Ausencia de controles de abuso	Sin rate limiting, timeouts homogéneos ni límite explícito de cuerpo	Límites, timeouts, cabeceras y observabilidad sin datos sensibles

Operación reproducible observada

Requisitos y configuración

- Node.js compatible con las dependencias declaradas; el repositorio conserva `bun.lock`, pero no fija versión de Node ni contiene CI.
- Variables referenciadas: `GEMINI_API_KEY`, `OAUTH_CLIENT_ID`, `NODE_ENV` y `DISABLE_HMR`.
- Los valores reales deben permanecer fuera de Git. `.env.example` solo sirve de inventario y no demuestra una configuración desplegada.

Construcción y ejecución local documentadas

```
npm install
npm run build
npm run start
```

`npm run build` genera el frontend Vite y `dist/server.cjs`; `npm run start` ejecuta el servidor empaquetado en el puerto `3000`. Estos comandos se derivan de `package.json`, pero la prueba previa se realizó sobre una copia temporal y sin invocar servicios externos.

Limitaciones de despliegue

No existen Dockerfile, manifiesto Cloud Run, healthcheck, volumen persistente, migración ni rollback versionados. Antes de trasladar PULA a DigitalOcean deben definirse imagen reproducible, almacenamiento, secretos, autenticación, proxy/TLS, backup/restore, monitorización y pruebas.

Riesgos y pendientes reales

Prioridad alta

- Proteger lecturas y mutaciones con autenticación y autorización.
- Bloquear SSRF en la importación manual.
- Sacar `data.json` y los contactos del repositorio; tratar los emails como datos personales.
- No marcar `sent` tras error o ausencia de Gmail.
- Definir persistencia compatible con Cloud Run y operaciones concurrentes.
- Aportar un despliegue reproducible que vincule imagen y runtime con un SHA Git.

Prioridad media

- Usar bloqueo y reemplazo atómico en vez de escrituras síncronas directas.
- Restringir CORS, limitar cuerpos y cargas, añadir rate limiting y cabeceras.
- Validar HTTP del scraper, fijar timeouts y evitar logs OAuth detallados.
- Añadir tests unitarios, API, scraper con fixtures, Gmail simulado y CI.
- Elegir y fijar un único gestor de dependencias y lockfile.

Deriva documental

- Arquitectura: React 18; manifiesto: React 19.
- Documentación: scraping ilimitado; código: cinco páginas y 40 enlaces.
- `AGENTS.md` reserva Gmail y URL manual para v2.0; ambos ya están en `main`.
- La etiqueta y los artefactos ErasmusHomes están depositados en el historial de PULA pese a pertenecer al repositorio independiente `ratienza/ErasmusHomes`.
- Regla visual: prohíbe `Ver descripción`; el scraper lo asigna a `size`.
- Cloud Run y Nginx declarados sin Dockerfile ni configuración reproducible.
- La automatización Python usa mock, OAuth local y `TEST_MODE = True`; no pertenece al runtime Express demostrado.

Pruebas y límites

-  Checkout Nexus limpio y sincronizado.
-  Instalación temporal, `tsc --noEmit` y build Vite/esbuild.
-  No existe suite automatizada.
-  No se llamó a SCPU, Gemini, Gmail, Firebase ni datos vivos.
-  Cloud Run no fue inspeccionado ni validado.
-  ErasmusHomes solo se consultó para confirmar la separación; no fue auditado.
-  No se ejecutaron los scripts `test-*`: son comprobaciones exploratorias con acceso a datos o servicios, no una suite aislada.

Operación futura y traslado

Antes de DigitalOcean se necesitan runtime reproducible, almacenamiento persistente, secretos fuera de Git, autenticación, backup/restore, healthcheck y tests. No deben reutilizarse datos o credenciales de Cloud Run implícitamente.

`Checkout ≠ Runtime` y `Tarjeta App Launch ≠ Runtime local`: PULA no se desplegó en Nexus ni en DigitalOcean; ambos launchers enlazan la producción pública existente. El runtime público no se tocó.

Aplicación · App Launch

Catálogo **multientorno** y capa de navegación para acceder a aplicaciones públicas y servicios internos. Se publica en Nexus y DigitalOcean; la presentación es común, pero cada entorno puede recibir un catálogo diferente. App Launch no es el runtime de las aplicaciones enlazadas.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / Nexus:** [App Launch interno](#)
- **Aplicación / producción:** [App Launch público](#)

Estado auditado

Campo	DigitalOcean	Nexus
Desarrollo	Codex · HTML/CSS/JavaScript	Mismo origen
Repositorio	ratienza/Apps_Lauch · main · 2d265ee1caf3c370e662cdc99cb923f4db457465	Mismo origen
URL	http://app.raulatienza.com y https://app.raulatienza.com	http://192.168.18.220
Publicación	Nginx del host · /opt/portal	Compose app-launch · Nginx 1.27-alpine
Puerto	80/443 , sin puerto explícito	80 → 80/tcp
Catálogo activo	catalogs/public.json → apps.json	catalogs/nexus.json → apps.json
Validación	HTTP/HTTPS, HTML, PNG y JSON: 200	HTML, PNG y JSON: 200 ; servicios enlazados sanos

Validación final: **21/08/2026**. GitHub y el checkout local coincidían en el SHA indicado; ambos catálogos desplegados respondieron **200** y conservaron exactamente el contenido versionado.

Arquitectura y funcionamiento

`index.html` carga `./apps.json` en tiempo de ejecución y crea las tarjetas mediante nodos DOM. Nombre, descripción y acción se asignan con `textContent` ; el manifiesto no se interpreta como HTML.

El despliegue publica solo `index.html` , el fondo y el catálogo seleccionado renombrado como `apps.json` . DigitalOcean no recibió `catalogs/nexus.json` , direcciones `192.168.18.220` , puertos `8081-8083` ni nombres exclusivos del laboratorio.

En Nexus el contenedor monta en solo lectura:

```
/home/raul/app-launch/site → /usr/share/nginx/html /home/raul/app-launch/nginx.conf → /etc/nginx/conf.d/default.conf
```

Usa una red Compose propia, no tiene persistencia funcional y se recupera con `restart: unless-stopped` .

Catálogos comprobados

El catálogo público contiene Reservas, Consumos Cupra, Multimedia VPalace, CV y PULA. El catálogo Nexus contiene Replicant Lab, Reservas, Multimedia VPalace y enlaces externos a Consumos, CV y PULA. En ambos casos PULA apunta a `https://pula-erasmus-housing-automator.ai.studio/` . Un enlace presente en un catálogo no implica que su aplicación se ejecute en el host del launcher.

EH-002 añade únicamente al catálogo Nexus una tarjeta interna **ErasmusHomes · Control del MVP**, dirigida a la página derivada dentro del runtime documental existente de Replicant Lab. El catálogo público y todas las tarjetas preexistentes permanecen sin cambios.

Regla

Tarjeta App Launch ≠ Runtime local.

Actualización y rollback

```
powershell -ExecutionPolicy Bypass -File .\deploy\deploy.ps1 -Target public powershell -ExecutionPolicy Bypass -File .\deploy\verify.ps1 -Target public powershell -ExecutionPolicy Bypass -File .\deploy\deploy.ps1 -Target nexus powershell -ExecutionPolicy Bypass -File .\deploy\verify.ps1 -Target nexus
```

La actualización permanente debe partir de `main`. Para rollback se revierte el cambio en Git mediante una rama/PR y se vuelve a desplegar el destino afectado; no se copian ficheros desde un host hacia GitHub.

Seguridad y límites

- El repositorio no contiene claves ni configuración local real; `deploy/local.settings.ps1` está ignorado.
- La portada pública no solicita contraseña. La autenticación pertenece a cada aplicación enlazada.
- Nexus es HTTP de LAN; no se declara HTTPS interno.
- El puerto `8080` está libre y no pertenece a App Launch.
- El despliegue público conserva algunos assets PWA históricos no versionados en `/opt/portal/assets`; no intervienen en la portada actual y quedan como deriva a limpiar solo mediante un cambio Git deliberado.

Pruebas realizadas

- Verificación versionada `public`: catálogo exacto, ausencia de referencias Nexus y HTTP/HTTPS correctos.
- Verificación versionada `nexus`: catálogo exacto y salud HTTP de `8081`, `8082` y `8083`.
- `nginx -t` correcto en DigitalOcean.
- Comprobación visual automatizada de escritorio y móvil incluida en el cierre documental.

ErasmusHomes · Control del MVP

Accesos

- **Panel de control:** [Control MVP autónomo](#)
- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / producción:** no hay una URL de producto ErasmusHomes desplegada.

Objetivo diciembre

Piloto comercial público antes del 20 de diciembre de 2026

Sincronizado

SHA ErasmusHomes main

fe867b64f53febfe7d203718b03c2e5dff31169c

Última sincronización

2026-08-23T09:00:09+02:00

[Abrir roadmap acordado \(DOCX\)](#)

Archivo:

Roadmap_ErasmusHomes_MVP_Diciembre_2026.docx

Fecha: 2026-08-22

Git: fe867b64 ·

[ver fuente versionada](#)

SHA-256: f4e6be71905f7ed234ebd88a84d9b482b6b464b7fdd7bc4ec7241c29d5cbf36b

12%

completado

2

Completado

0

En ejecución

15

Pendiente

0

Bloqueado

Próximo gate

W-2026-08-31 · Supply y validación

Retirar toda ciudad sin tres fuentes viables; no forzar scraper.

Esta semana

No hay tareas en ejecución.

Kanban

Completado · 2

En ejecución · 0

Pendiente · 15

Bloqueado · 0

EH-001

Consolidar la fuente de verdad

Objetivo: Consolidar fuentes, decisiones y alcance en Git y sincronizar Nexus.

Terminado: PR fusionado en main y GitHub, local y Nexus en el mismo SHA.

Riesgos: Divergencia histórica de PDFs ya documentada

[Evidencia](#)

EH-002

Roadmap canónico y panel Replicant Lab

Objetivo: Publicar roadmap estructurado y panel interno generado sin duplicar el estado.

Terminado: PRs validados, panel derivado y sincronización documentada sin nuevo servicio.

Riesgos: Integración coordinada entre tres repositorios

[Evidencia](#)

W-2026-08-31

Supply y validación

Objetivo: Evaluar hasta tres ciudades, fuentes, permisos, calidad, volumen y cinco entrevistas.

Terminado: Ficha por ciudad y decisión de piloto con tres fuentes viables por ciudad elegida.

Riesgos: Fuentes insuficientes o no permitidas

Evidencia pendiente

W-2026-09-07

Arquitectura y confianza

Objetivo: Definir datos, estados, eventos, permisos, privacidad, retención y Gmail fallback.

Terminado: ADR técnico, modelo de datos y threat checklist aceptados.

Riesgos: OAuth restringido, Sobrearquitectura

Evidencia pendiente

W-2026-09-14

UX móvil y prototipo

Objetivo: Diseñar onboarding, hoy, resultados, detalle, shortlist, contacto y seguimiento a 360-430 px.

Terminado: Prototipo navegable probado con 3-5 personas.

Riesgos: Flujos contemplativos sin acción

Evidencia pendiente

W-2026-09-21

Base PWA

Objetivo: Preparar CI, entornos, auth, perfil, PWA instalable, diseño y observabilidad mínima.

Terminado: La app abre en móvil y persiste el perfil de un usuario.

Riesgos: Incompatibilidad móvil o persistencia

Evidencia pendiente

W-2026-09-28

Ingesta y listings

Objetivo: Integrar primeras fuentes permitidas con normalización, deduplicación y origen.

Terminado: Listings reales o dataset trazable disponible en una ciudad.

Riesgos: Fuente bloqueada, Datos duplicados

Evidencia pendiente

W-2026-10-05

Búsqueda y selección

Objetivo: Implementar filtros, matching explicable, shortlist y estados.

Terminado: Un usuario obtiene una shortlist útil en menos de cinco minutos.

Riesgos: Matching opaco o sin evidencia

Evidencia pendiente

W-2026-10-12

Contacto seguro

Objetivo: Crear plantillas multilingües editables, consentimiento y derivación controlada.

Terminado: Mensaje contextual listo y acción auditada.

Riesgos: Spam, Mensaje incorrecto

Evidencia pendiente

W-2026-10-19

Seguimiento

Objetivo: Añadir estados, recordatorios, forwarding o vinculación de hilos y pendientes.

Terminado: Dashboard de qué hacer hoy operativo.

Riesgos: Auditoría OAuth retrasa el piloto

Evidencia pendiente

W-2026-10-26

Conversación y alertas

Objetivo: Resumir hilos, extraer peticiones y sugerir alertas y siguiente acción.

Terminado: Tres conversaciones de prueba interpretadas correctamente.

Riesgos: Interpretación errónea de IA

Evidencia pendiente

W-2026-11-02

Vertical slice

Objetivo: Completar buscar, seleccionar, contactar y seguir con datos reales.

Terminado: Demo móvil reproducible de punta a punta.

Riesgos: Bloqueantes P0/P1

Evidencia pendiente

W-2026-11-09

Calidad y cumplimiento

Objetivo: QA móvil, accesibilidad, seguridad, privacidad, logs, errores y recuperación.

Terminado: Checklist de release y registro de riesgos aceptados.

Riesgos: Privacidad, Accesibilidad, Recuperación incompleta

Evidencia pendiente

W-2026-11-16

Piloto cerrado

Objetivo: Incorporar primeros usuarios, prestar soporte ligero y medir activación/contacto.

Terminado: Primeros usuarios reales y panel de métricas.

Riesgos: Soporte fragmentado, Muestra insuficiente

Evidencia pendiente

W-2026-11-23

Corrección por evidencia

Objetivo: Resolver los tres mayores frenos observados y validar fuentes y mensajes.

Terminado: Release candidate pública basada en evidencia.

Riesgos: Cambios sin evidencia, Scope creep

Evidencia pendiente

W-2026-11-30

Lanzamiento piloto

Objetivo: Publicar PWA, monitorizar, soportar y comunicar cobertura honestamente.

Terminado: Piloto comercial público estable y observable.

Riesgos: Inestabilidad, Cobertura mal comunicada

Evidencia pendiente

W-2026-12-07

Estabilización y decisión de enero

Objetivo: Corregir incidencias, medir embudo, entrevistar y decidir el backlog de enero.

Terminado: Informe de piloto y backlog 2027 priorizado.

Riesgos: Escalado prematuro

Evidencia pendiente

Línea temporal hasta el 20 de diciembre

2026-08-22 - 2026-08-30	Consolidar la fuente de verdad	Completado
2026-08-22 - 2026-08-30	Roadmap canónico y panel Replicant Lab	Completado

2026-08-31 - 2026-09-06	Supply y validación	Pendiente
2026-09-07 - 2026-09-13	Arquitectura y confianza	Pendiente
2026-09-14 - 2026-09-20	UX móvil y prototipo	Pendiente
2026-09-21 - 2026-09-27	Base PWA	Pendiente
2026-09-28 - 2026-10-04	Ingesta y listings	Pendiente
2026-10-05 - 2026-10-11	Búsqueda y selección	Pendiente
2026-10-12 - 2026-10-18	Contacto seguro	Pendiente
2026-10-19 - 2026-10-25	Seguimiento	Pendiente
2026-10-26 - 2026-11-01	Conversación y alertas	Pendiente
2026-11-02 - 2026-11-08	Vertical slice	Pendiente
2026-11-09 - 2026-11-15	Calidad y cumplimiento	Pendiente
2026-11-16 - 2026-11-22	Piloto cerrado	Pendiente
2026-11-23 - 2026-11-29	Corrección por evidencia	Pendiente
2026-11-30 - 2026-12-06	Lanzamiento piloto	Pendiente
2026-12-07 - 2026-12-20	Estabilización y decisión de enero	Pendiente

La fuente canónica es `ratienza/ErasmusHomes/docs/board/roadmap.yaml` . Esta página es un artefacto derivado.

Aplicación · Salones AV

Documentación operativa audiovisual para el personal del SH Valencia Palace: panel general, pantallas LED, traslado, conexión de cliente, sonido y mapas de las plantas PL1 y PL6.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / Nexus:** [Salones AV](#)

Ficha de infraestructura

Campo	Valor
Desarrollo	Codex / GitHub
Repo	<code>ratienza/salones-av-valencia-palace</code>
Host	Nexus
Ruta	<code>/opt/apps/salones-av-valencia-palace</code>
Contenedor	<code>salones-av</code>
Imagen	<code>nginx:alpine</code>
Puerto	<code>192.168.18.220:8081 → 80/tcp</code>
Contenido	<code>proyecto_html</code> montado en solo lectura
Rama / SHA	<code>main · 8c0bc08446256974b7efa57c730cbeb1b6e81520</code>
Red	Compose propia, sin dependencias con otros contenedores
Inicio	<code>restart: unless-stopped</code>

Comportamiento de despliegue

El contenido HTML se sirve mediante un bind mount. Por tanto, un `git pull` actualiza los ficheros visibles sin necesidad de reconstruir una imagen.

La actualización segura exige revisar primero el working tree, actualizar desde `main` y comprobar enlaces y páginas. El rollback consiste en volver a un commit aprobado y restaurar el contenido estático; no tiene base de datos ni persistencia funcional.

Reconciliación Git y Nexus

La Fase 2A corrigió mediante el PR `salones-av-valencia-palace#2` la única deriva del checkout: `main` ahora versiona `192.168.18.220:8081:80`. El cambio se reconstruyó conscientemente en una rama; no se copió el checkout del servidor hacia GitHub.

Antes del avance rápido se demostró que el blob preparado, el archivo vivo y el stash preservativo eran idénticos: `686c added506 added67b624b addedfae5f49ca640a799da6`. Después del merge, GitHub `main`, `origin/main` y el checkout Nexus quedaron en `8c0bc08446256974b7efa57c730cbeb1b6e81520`, sin cambios locales.

Validación del 13/08/2026

- Contenedor `salones-av` activo con imagen `nginx:alpine`.
- `http://192.168.18.220:8081/` respondió `200`.
- Las siete páginas del menú respondieron `200`.
- Los enlaces HTML locales pasaron la comprobación estática.
- `docker compose config --quiet` aceptó la definición versionada.
- Docker confirmó el bind efectivo `192.168.18.220:8081`; no existe publicación de Salones en `0.0.0.0:8081`.
- GitHub `main == checkout Nexus` en `8c0bc08` tras el despliegue dirigido al servicio `web`.
- La copia pública `https://app.raulatienza.com/salones/` respondió `200`.
- No se validaron equipos AV físicos, números de tomas ni procedimientos sobre hardware.

Seguridad y límites

El contenido operativo puede incluir topología e inventario de salas. No deben incorporarse credenciales ni datos de red sensibles a un repositorio público. El servicio es estático y no implementa autenticación propia; su alcance depende del bind LAN y del control del host.

Alcance de esta ficha

La documentación técnica/funcional específica de los salones permanece en el repo de la aplicación.

Reserva-Pistas-UTP

Ficha de infraestructura y operación de la aplicación de reservas de pádel de Torre de Porta-Coeli. La documentación funcional y de desarrollo permanece en [ratienza/Reserva-Pistas-UTP](#) ; esta página registra el montaje real en Raul Lab y producción.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / producción:** [Reserva-Pistas-UTP](#)
- **Aplicación / Nexus:** [Runtime interno](#)

Estado validado

Elemento	Valor
Desarrollo	Codex / GitHub
Repositorio	ratienza/Reserva-Pistas-UTP
main validado	6fb055e16eb232d47cc2d759bed0ce2caff4cec9 (runtime) + 6df1698d3346db5c35f7d173b5d7dc2567ce8a5e (guía operativa)
Nexus	✅ Docker Compose operativo
Ruta / URL Nexus	/opt/apps/Reserva-Pistas-UTP · http://192.168.18.220:8083
Datos Nexus	/opt/data/reserva-pistas → /app/data
DigitalOcean	✅ reserva-pistas.service en loopback:8765
Ruta / URL producción	/opt/reserva-pistas · https://app.raulatienza.com/padel/
Publicación	Nginx + HTTPS + HTTP Basic existente
Sincronización	✅ Bidireccional, por registros y exclusivamente bajo demanda
Issue de handover	#2 cerrado con evidencias

Validación de sincronización: **10/08/2026**. Revalidación de servicio y código: **13/08/2026**. No se cambiaron puertos, UFW, DNS, certificados, Nginx, autenticación pública ni otros servicios.

Arquitectura real

```
Nexus · 192.168.18.220:8083 reserva-pistas-nginx ↓ red privada Docker reserva-pistas-app · app:8765 · UID/GID 1000 ↓ /app/data ↕
/opt/data/reserva-pistas | | SSH dedicado: restrict + comando forzado | solo ping/export/apply/backups/restore ↕ DigitalOcean ·
app.raulatienza.com/padel/ Nginx · HTTPS · HTTP Basic ↓ reserva-pistas.service · reserva:reserva loopback · puerto 8765 ↓ /opt/reserva-
pistas
```

Nexus coordina ambas direcciones. La clave dedicada está montada de solo lectura, la huella del host DigitalOcean fue contrastada con el canal SSH administrativo ya confiable y el `authorized_keys` remoto no concede shell, TTY ni forwarding. El comando forzado baja privilegios a `reserva` antes de ejecutar `sync_peer.py`.

Datos y exclusiones

Estado	Clasificación	Sincronización
Registros de <code>tasks.local.json</code>	Funcional e histórico	Sí, por <code>id</code>
<code>_sync</code> por registro	Origen, creación, modificación, revisión, tombstone y hash	Sí
<code>credentials.local.json</code>	Credencial local	Nunca
<code>notifications.local.json</code>	Token/configuración local	Nunca
<code>telegram.users.local</code>	Autorizaciones locales	Nunca
<code>telegram.offset.local</code> , <code>telegram.state.local</code>	Estado efímero	Nunca
<code>sync-state.local.json</code>	Base de comparación Nexus	No
<code>sync-audit.local.jsonl</code>	Auditoría sin valores privados	No
<code>backups/</code> , <code>logs</code> , <code>PID</code> , <code>red</code> y <code>app.local.env</code>	Backup/configuración/estado local	Nunca

La migración idempotente conservó los 18 registros y eliminó `username` y `password` del histórico. Las tareas consultan `credentials.local.json` únicamente al ejecutarse. `/api/settings` ya no devuelve la contraseña al navegador.

Runtime revalidado el 13/08/2026

Elemento	Evidencia
GitHub / Nexus	<code>main</code> · <code>6df1698d3346db5c35f7d173b5d7dc2567ce8a5e</code>
Contenedores	<code>reserva-pistas-nginx</code> y <code>reserva-pistas-app</code> activos
Publicación Nexus	Nginx <code>192.168.18.220:8083</code> → <code>80</code> ; backend sin puerto host
Persistencia	<code>/opt/data/reserva-pistas</code> → <code>/app/data</code> en lectura/escritura
Usuario de app	UID/GID <code>1000:1000</code>
Autoarranque	<code>restart: unless-stopped</code>
DigitalOcean	<code>reserva-pistas.service</code> activo como <code>reserva:reserva</code>
Producción	<code>/padel/</code> devolvió <code>401</code> sin credenciales, como se esperaba

Los hashes de `app.py`, `templates/index.html` y `sync_engine.py` desplegados en DigitalOcean coincidieron con `main`. Se ejecutaron **21 pruebas** locales sobre una copia limpia y todas terminaron correctamente. No se hicieron reservas, cancelaciones, sincronizaciones, notificaciones ni escrituras sobre datos reales.

Fusión y conflictos

El motor valida JSON e identificadores, calcula hashes canónicos por registro y compara cada lado con la base de la última sincronización:

1. incorpora registros presentes solo en el origen;
2. propaga cambios únicamente del origen;
3. conserva e informa cambios únicamente del destino;
4. conserva registros presentes solo en el destino;
5. propaga cancelaciones y tombstones explícitos;

6. si el mismo `id` cambió en ambos lados, no escribe hasta elegir Nexus, DigitalOcean o cancelar;
7. propaga el lado elegido por la persona;
8. una segunda simulación sin cambios no escribe ni crea un backup innecesario.

Los conflictos visibles se limitan a campos operativos no sensibles. Cada escritura usa bloqueo entre procesos, fichero temporal, `fsync`, reemplazo atómico, verificación de la huella simulada y backup previo.

Tareas activas

El destino no se modifica mientras tenga tareas `queued` o `running`. Si una tarea activa del origen se incorpora al otro servidor, la copia queda neutralizada como `cancelled` / `sync_neutralized`; conserva el histórico pero no inicia un segundo ejecutor. No existe sincronización automática ni periódica.

Panel administrativo

En **Ajustes** → **Sincronización administrativa** aparecen:

- DigitalOcean → Nexus ;
- Nexus → DigitalOcean ;
- estado del canal seguro;
- simulación con nuevos, actualizados, sin cambios, cancelaciones, conflictos, tareas activas y backup previsto;
- resolución humana por `id` ;
- confirmación explícita, protección contra doble clic y una sola operación concurrente;
- listado y restauración confirmada de backups.

En Nexus todo el backend usa Basic Auth local guardado fuera de Git. En DigitalOcean se conserva el Basic Auth de Nginx. Si falta conectividad o permisos, los botones quedan deshabilitados.

Backups y recuperación

Backups verificados:

Servidor	Backup	Resultado
DigitalOcean	predeploy-20260810T014853+0200	Datos, configuración, código, plantilla y unidad systemd; JSON y manifiesto SHA-256 válidos
Nexus	tasks.local.json.20260810T015121+0200.backup	Estado vacío previo a la primera escritura; JSON válido y restaurable

Restaurar desde el panel exige confirmación y crea antes otro backup del estado reemplazado. Backups, credenciales y logs nunca atraviesan el canal de sincronización.

Primera sincronización comprobada

Paso	Resultado real
DigitalOcean antes de migrar	18 registros: 12 cancelados, 6 reservados, 0 activos, 0 duplicados
Migración de secretos	Sin credenciales incrustadas; hash funcional anterior y posterior idéntico
Simulación DigitalOcean → Nexus	18 nuevos, 12 cancelaciones históricas, 0 conflictos, 0 activos
Simulación Nexus → DigitalOcean	18 solo en destino, 0 conflictos; producción no se escribió
Aplicación	Solo DigitalOcean → Nexus

Paso	Resultado real
Segunda simulación, ambas direcciones	18 sin cambios, 0 nuevos, 0 actualizados, 0 conflictos
Hash normalizado común	db9a806429479d9e83c83724ca67b5e072ca2ab29b94fbde9dbd19475342dc09
Configuración local	Credenciales y notificaciones de producción conservaron hash; Nexus no recibió secretos ni estado Telegram

Pruebas y entrega

- 21 pruebas: cambios unilaterales, altas en ambos lados, conflictos, cancelaciones, duplicados, repetición, interrupción, JSON inválido, conectividad, tareas activas, confirmación y restauración.
- Compilación Python y sintaxis JavaScript.
- CI con build Docker y ejecución real como UID/GID 1000.
- Nexus: build, Compose, HTTP 401/200, panel, persistencia, responsive CSS y logs sin errores.
- DigitalOcean: systemd, backend, Nginx, HTTP 401 público, migración equivalente y logs sin warnings.
- PR funcional [#4](#), corrección de runtime [#5](#) y guía validada [#6](#), todos fusionados mediante squash.
- El PR borrador #1 quedó cerrado al quedar incorporado y reconciliado en #4.

Operación

```
# Nexus cd /opt/apps/Reserva-Pistas-UTP git switch main git pull --ff-only origin main docker compose up -d --build # Estado curl -I
http://192.168.18.220:8083/ docker compose ps # Producción systemctl is-active reserva-pistas nginx nginx -t journalctl -u reserva-pistas -n
100 --no-pager
```

No ejecutar reservas ni enviar notificaciones para probar el handover. Las comprobaciones de datos se realizan con simulación, hashes, recuentos e identificadores normalizados.

Aplicación · Consumos Cupra

Aplicación personal para registrar y analizar consumos de combustible. Fue creada inicialmente con AI Studio y remediada con Codex sin alterar su funcionalidad.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / producción:** [Consumos Cupra](#)

Estado actual

Campo	Valor
Repositorio	ratienza/Consumos_Cupra · privado
main desplegado	9f66a368a1dfd58e5b52741e263e77866397a7f7
Build	7db2eddf-8b54-46fa-a04b-1abcee75d72c
Imagen	cosnumos-cupra:9f66a368...
Digest	sha256:df935d3ace3ce6a319094dccfbae80606bc4a3c6d3044f87251d350443be8ed5
Runtime	Google Cloud Run · revisión cosnumos-cupra-00009-pon · tráfico 100 %
Rollback disponible	cosnumos-cupra-00007-b7b
Nexus / DigitalOcean	Sin runtime canónico; App Launch únicamente enlaza la aplicación

Arquitectura y despliegue

AI Studio / Codex → GitHub → Cloud Build → Artifact Registry → Cloud Run

React 19, Vite, Tailwind y Recharts forman el frontend. Express sirve el bundle y las APIs de registros, configuración, pendientes y diagnóstico. Google Sheets/Apps Script, Gemini y webhooks son dependencias externas opcionales; sus credenciales permanecen fuera de Git.

El trigger productivo construye desde GitHub y crea una revisión candidata con `--no-traffic`, label `candidate`, `entrypoint: gcloud` y `CLOUD_LOGGING_ONLY`. La promoción de tráfico es un paso controlado posterior.

Validación y operación

La revisión activa se validó con HTTP 200 en raíz, manifest, health, APIs GET no destructivas, configuración, pendientes, iconos, JavaScript y CSS. El build reproducible usa el lockfile y `npm ci`; lint, build e imagen Docker quedaron validados antes de la promoción.

El rollback consiste en devolver el tráfico a `cosnumos-cupra-00007-b7b` o desplegar de nuevo su imagen conocida. No requiere reconstruir el código ni intervenir DigitalOcean o Nexus.

Seguridad y deuda

- No mostrar valores de variables, secretos o integraciones externas.
- Los pushes a ramas conectadas pueden activar Cloud Build; los cambios productivos requieren revisar trigger, candidato y tráfico.
- `npm audit` registra cuatro vulnerabilidades conocidas: una baja, una moderada y dos altas. No se ejecutó `npm audit fix`; su tratamiento debe hacerse en un cambio separado y probado.

Aplicación · CV de Raúl

Currículum y portfolio profesional público creado con AI Studio, con frontend Vite/Tailwind y descarga de PDF generada durante la compilación.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / producción:** [CV de Raúl](#)

Estado actual

Campo	Valor
Repositorio	ratienza/CV-Raul-IA-Estudio-Google- · privado
main	0da08cfa98e5ad9e5e81ed48ac4cd4428360d618
Producción	Firebase Hosting · proyecto replicant-lab
Versión activa observada	092bfc
Backend público identificado	cv-backend
URL	https://cv.raulatienza.com · HTTP 200
Nexus	Checkout limpio de consulta; sin contenedor, servicio o puerto

Arquitectura real

La cadena demostrada actualmente es:

despliegue manual Firebase Hosting → infraestructura Google/Firebase → cv.raulatienza.com

No está demostrada una cadena automática GitHub → producción ni la relación exacta entre el SHA actual y la versión Firebase 092bfc .

El servicio Cloud Run cv-raulatienza-com del proyecto consumos-cupra , región europe-west1 , conserva una revisión placeholder y un trigger fallido. Es residual y no constituye la producción pública. La reescritura Cloud Run declarada en firebase.json tampoco corresponde al servicio público observado.

Build, validación y rollback

El proyecto usa Vite, Tailwind, Node 20 y una generación PDF con PDFKit. El Dockerfile disponible construye con Node y sirve el resultado mediante Nginx, pero no describe el runtime canónico actual.

Producción, /health y /manifest.json respondieron 200 ; las dos últimas rutas son fallback de la SPA, no healthcheck ni manifest independientes. Firebase conserva versiones anteriores, por lo que existe capacidad de rollback desde Hosting.

Seguridad y deuda POST-CARTERA

No existe un problema crítico actual. Queda aceptado para después de Cartera:

- reconciliar Firebase, Cloud Build y configuraciones residuales;
- retirar o corregir el trigger obsoleto y demostrar trazabilidad SHA → versión Firebase;
- lockfile, reproducibilidad, CI, tests y typecheck;
- restos React, metadata Gemini, artefactos residuales, cabeceras, caché, protección de `main` y UX de revelado de contacto.

El checkout de Nexus no debe ejecutarse ni utilizarse para desplegar o restaurar producción.

Aplicación · Control de Red

Panel PowerShell para inventariar, nombrar y revisar dispositivos de la red local desde Replicant/Windows.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Runtime:** local en Replicant · sin URL publicada.

Estado auditado

Campo	Valor
Desarrollo	PowerShell + Codex
Repositorio	ratienza/control-red · privado
main	0e285d26f10ccb58e58d3ebbef35379b00a4b41d
Replicant	Herramienta local; no se ejecutó un escaneo durante la auditoría
Nexus	Checkout en /opt/apps/control-red ; sin contenedor, servicio o puerto
Entrada	ABRIR_PANEL.cmd → panel-control-red.ps1
Persistencia	Inventario JSON y snapshots versionados en el repositorio actual

Funcionalidad comprobada

El código implementa una interfaz PowerShell para descubrimiento, inventario, alias y snapshots de red. La sintaxis completa del script se analizó correctamente sin ejecutarlo, evitando escaneos o cambios sobre dispositivos.

Operación y rollback

La herramienta debe ejecutarse desde Replicant, donde existen PowerShell y acceso a la LAN. El checkout de Nexus es solo una copia de código/datos y no convierte el panel en aplicación web.

El rollback de código consiste en volver a un commit conocido mediante rama/PR. Los inventarios y snapshots no deben reemplazarse ni eliminarse automáticamente: requieren copia y revisión específica por contener estado del entorno.

Seguridad y pendientes

- El repositorio privado contiene inventario y snapshots reales versionados. Esto contradice el criterio global de mantener datos vivos fuera de Git, aunque su visibilidad sea privada.
- No se exponen aquí direcciones MAC, nombres de personas ni detalles del inventario.
- Pendiente: separar datos operativos del código, añadir ejemplos anonimizados y documentar backup/recuperación antes de cualquier limpieza.
- No está desplegado ni validado como servicio en Nexus.

Aplicación · Cartera Estratégica

Aplicación desarrollada con Codex y Python/Streamlit para operación local en Replicant.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Runtime:** local en Replicant · sin URL publicada.

Estado

Aplicación Python/Streamlit operativa como MVP local en Windows, con versión documental `v1.2.0` y SQLite privada fuera del repositorio. GitHub `main` estaba en `f2b319dea862469b90eb65fa20f8ae2d8c96875d` ; el checkout de trabajo observado en Replicant estaba atrasado en `3be6254` y no se modificó.

Entorno comprobado

El entorno vigente es Replicant/Windows con ejecución local de Streamlit. No existe en la documentación actual de la aplicación una decisión aprobada que convierta Nexus o Docker/Linux en su siguiente destino.

Consideraciones

- La base privada no debe entrar en Git.
- Tokens como EODHD deben inyectarse como secreto/entorno.
- Los datos de inversión pueden permanecer solo en local.
- Cualquier soporte Docker/Linux o despliegue remoto deberá decidirse e implementarse en el repositorio de la aplicación mediante rama y PR.

Arquitectura y funciones verificadas

El núcleo usa Python, SQLite y migraciones versionadas; Streamlit es la presentación. El repositorio incluye importación y conciliación, transacciones, FIFO, cash, backups, exportaciones, datos de mercado EODHD, estrategia 7A e inteligencia SMC 7B. Las credenciales se inyectan por entorno y la base, los backups, XLSX y exportaciones reales quedan fuera de Git.

La actualización debe hacerse únicamente sobre una copia y con backup SQLite verificado antes de migrar. El rollback usa el servicio de restauración y una copia probada; nunca se sustituye la base privada por un fichero del repositorio.

Pruebas del 13/08/2026

- `pytest --collect-only` : 369 pruebas recogidas.
- La suite aislada avanzó aproximadamente hasta el 58 % sin fallos antes del límite de cinco minutos.
- No se declara la suite completa como validada.
- No se abrió, copió, migró ni modificó la SQLite privada.

Pendiente

No está desplegada ni validada en Nexus. Tampoco debe presentarse Nexus como objetivo acordado hasta que exista una decisión versionada en el repositorio de la aplicación.

Aplicación · Replicant Lab

Documentación canónica del laboratorio: arquitectura, hosts, red, despliegues, operación, decisiones, pendientes y fichas técnicas de aplicaciones.

Accesos

- **Ficha técnica:** [HTML autocontenido](#)
- **Aplicación / Nexus:** [Replicant Lab](#)

Estado auditado

Campo	Valor
Desarrollo	Codex / MkDocs / Mermaid
Repositorio	<code>ratienza/infra-replicant-lab · main</code>
Punto de partida del cierre	<code>0134f5bdf932128de47545a03acab593027f797f</code>
Nexus	<code>/opt/apps/infra-replicant-lab · checkout limpio observado</code>
Servicio	<code>Compose docs · contenedor infra-replicant-docs</code>
Imagen	<code>infra-replicant-docs:local</code> , construida desde el <code>Dockerfile</code>
URL / puerto	<code>http://192.168.18.220:8082 · 192.168.18.220:8082 → 80/tcp</code>
Reinicio	<code>restart: unless-stopped</code>

Arquitectura y funcionamiento

Markdown, `mkdocs.yml` y los recursos de `docs/` son la fuente de verdad. El pipeline calcula una huella SHA-256, construye MkDocs en modo estricto y genera el dossier global y las fichas HTML/PDF individuales.

El Dockerfile usa dos etapas: Python/MkDocs construye el sitio y Nginx `1.27.4-alpine` sirve únicamente el resultado estático. El runtime no usa `mkdocs serve`, bind mounts, base de datos ni almacenamiento persistente.

Actualización y rollback

```
cd /opt/apps/infra-replicant-lab git switch main git pull --ff-only origin main docker compose up -d --build
```

La actualización se realiza solo después de integrar el PR. Para rollback se revierte el cambio mediante GitHub, se actualiza de nuevo `main` y se reconstruye exclusivamente este servicio; no se recupera código desde el contenedor.

Pruebas realizadas

- `python scripts/docs_pipeline.py generate --screenshots`: correcto.
- `python scripts/docs_pipeline.py check`: sincronía correcta.
- 32 páginas fuente y seis diagramas Mermaid validados por el pipeline.

- Dossier global de 52 páginas y ocho fichas individuales HTML/PDF sincronizadas.
- Runtime Nexus observado con respuesta 200 el 13/08/2026; la nueva versión se valida de nuevo tras el merge.

Seguridad, dependencias y pendientes

- No almacena secretos, datos vivos ni backups; GitHub contiene solo documentación y configuración versionable.
- La tipografía remota del tema es opcional: una caída de Google Fonts no bloquea el contenido ni los recursos locales.
- Depende de GitHub para actualización y de Docker para reconstrucción; no depende de las aplicaciones documentadas para servir el sitio.
- Pendiente transversal: definir y probar la política de backup/restauración de Nexus. La documentación está en Git, pero eso no sustituye los backups de datos de otras aplicaciones.

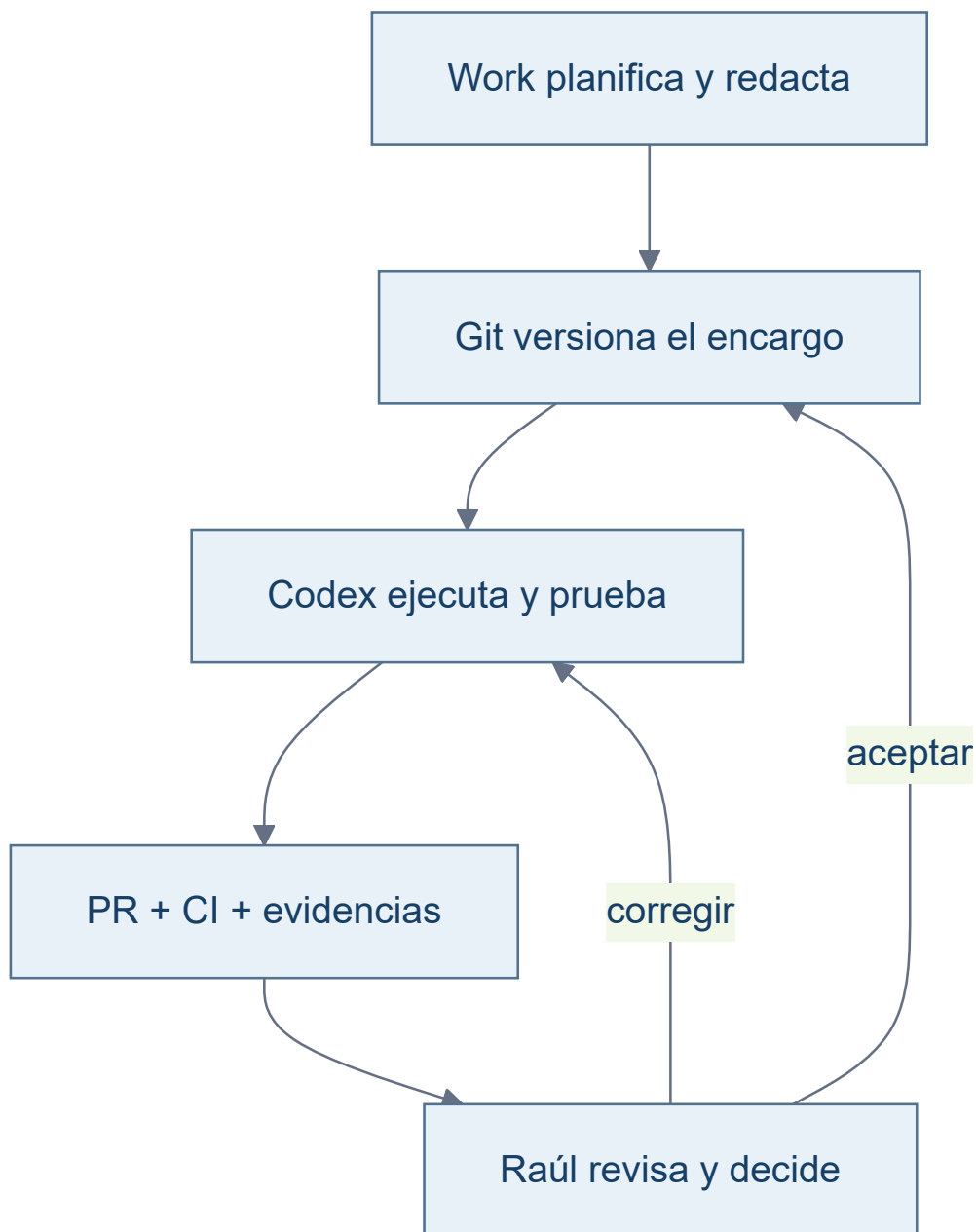
Gobierno del trabajo

Este contrato permite retomar un proyecto desde cualquier ordenador o conversación sin depender de memoria de chat. GitHub conserva contrato, implementación y evidencias; el runtime demuestra únicamente lo desplegado.

Roles y fuentes de verdad

Actor	Responsabilidad
Raúl · Product Owner	Prioriza, autoriza riesgos, acepta y decide merge/despliegue.
ChatGPT Work · Project Manager	Planifica, redacta encargos, revisa pruebas y prepara decisiones.
Codex · Ejecución	Implementa en la rama indicada, prueba, documenta y publica evidencias.
GitHub + CI	Conserva la verdad técnica y ejecuta verificaciones automáticas.
Runtime	Evidencia la versión realmente desplegada; nunca sustituye a Git.

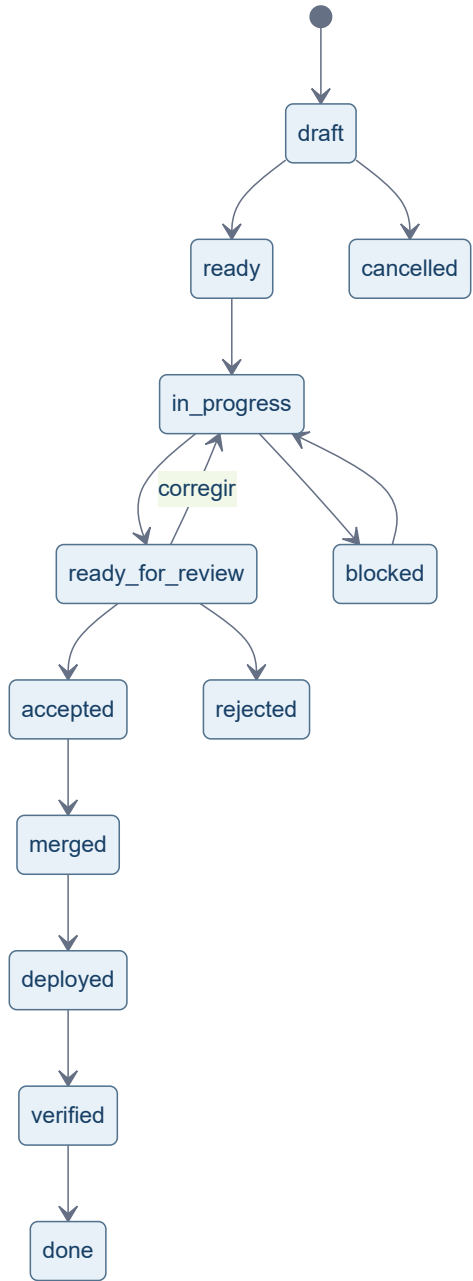
Jerarquía operativa: instrucciones del Proyecto de Work → AGENTS.md aplicable → roadmap/estado canónico → docs/encargos/<ID>.md → rama y PR → evidencias → merge → despliegue → verificación. Una capa no autoriza silenciosamente la siguiente.



Ciclo y estados

Planificación → encargo en Git → ejecución → PR y evidencias → revisión → corrección o aceptación → merge autorizado → despliegue autorizado → verificación → siguiente encargo. Una fase puede agrupar varios encargos, pero cada encargo lógico usa una rama propia.

Estados permitidos: `draft`, `ready`, `in_progress`, `ready_for_review`, `accepted`, `merged`, `deployed`, `verified`, `done`, `blocked`, `rejected` y `cancelled`.



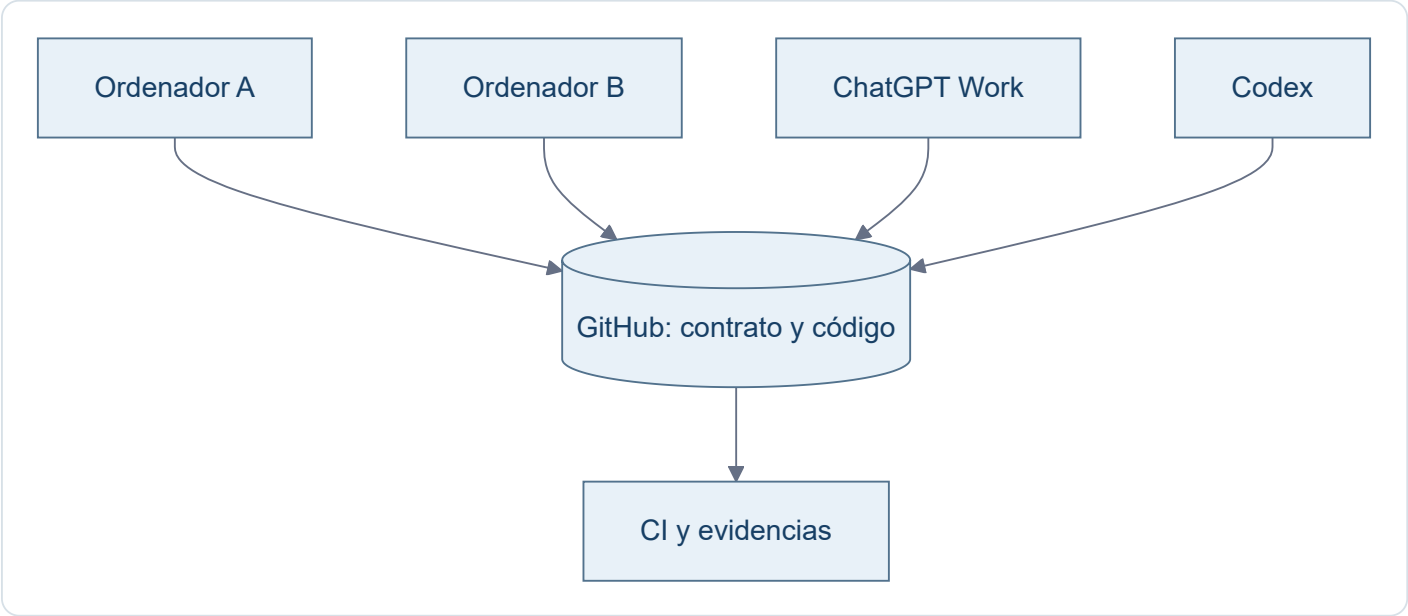
Evidencia	Significado exacto
Rama	Trabajo aislado; no está en <code>main</code> .
PR	Propuesta revisable; no está fusionada.
CI verde	Pruebas automáticas superadas; no es aceptación humana.
<code>main</code>	Historia canónica fusionada; no acredita despliegue.
Runtime	Versión observable en un entorno; debe identificarse con SHA.
Verificación	Comprobación posterior al despliegue de SHA, salud y comportamiento.

Chats y varios ordenadores

- Nuevo encargo: nueva conversación de Codex.

- Corrección del mismo encargo: misma conversación y rama.
- Nuevo resultado importante de Work: chat separado dentro del Proyecto.
- Cambio de `AGENTS.md` : reiniciar Codex antes del siguiente encargo.

Para retomar: entrar en el repositorio correcto, comprobar árbol limpio, hacer `fetch` , actualizar la base sin destruir cambios, iniciar Codex desde la raíz, indicar proyecto e ID exactos y verificar que se cargaron `AGENTS.md` y el encargo.



Coordinación multirrepositorio

El encargo enumera repositorios afectados y no afectados. Para cada uno registra rama, PR, SHA, CI y estado individual; además declara dependencias, orden de merge/despliegue y rollback. Nunca se resume todo como “verde” si un repositorio permanece pendiente.

Matriz de impacto documental

Cambio	Actualización obligatoria
Estado de encargo	Roadmap, board o estado canónico
Funcionalidad	README y documentación de usuario
Arquitectura	ADR y diagramas
Despliegue	Runbook, URL, versión y SHA
Varios repositorios	Manifiesto coordinador
Hito aceptado	Changelog y cierre
Regla recurrente	<code>AGENTS.md</code>

“Automático” significa actualizar solo la documentación afectada y comprobar coherencia cuando sea posible; no regenerar indiscriminadamente todo.

Definición de terminado

Un encargo termina cuando cumple criterios, pruebas y documentación; sus evidencias son accesibles; la aceptación está registrada; y, cuando corresponda, merge, despliegue y verificación tienen SHA y resultado propios. `ready_for_review` no equivale a `accepted` .

Checklist de reanudación

- [] Repositorio y remoto correctos.
- [] Árbol limpio y `origin/main` actualizado.
- [] `AGENTS.md` y encargo exacto leídos.
- [] Rama, SHA base, estado y autorizaciones confirmados.
- [] Dependencias multirrepositorio identificadas.
- [] Último PR, CI, evidencias y runtime contrastados por separado.
- [] Siguiente gate y siguiente acción explícitos.

Antipatronos prohibidos

No usar “ejecuta lo último”; guardar el contrato solo en chat; editar `main` ; reutilizar ramas ajenas; mezclar repositorios; confundir CI con aceptación; afirmar despliegue por existir un merge; declarar verificación sin observar runtime; cerrar aceptación visual con capturas locales; ni resolver colisiones silenciosamente.

Consulta también el [sistema de encargos](#) y el [bloque para Work](#).

Instrucciones para el Proyecto de Work

Bloque listo para copiar en las instrucciones de un Proyecto de ChatGPT Work. Versionarlo aquí no modifica por sí solo la configuración de Work.

Raúl es Product Owner y autoridad final. ChatGPT Work actúa como Project Manager: prepara contratos de encargo, mantiene la planificación, revisa evidencias y solicita aceptación. Codex ejecuta cambios, pruebas y documentación. GitHub es la fuente de verdad técnica y documental; CI verifica, pero no acepta. El runtime solo demuestra lo realmente desplegado. Cada trabajo debe tener un identificador exacto y un contrato versionado en docs/encargos/<ID>.md con estado permitido. Work no ordenará “ejecuta lo último”: indicará proyecto e ID. Antes de ejecución comprobará repositorios afectados, SHA base, rama, alcance, criterios, permisos y matriz documental. Tras el PR revisará CI y evidencias accesibles antes de proponer aceptación. Estados: draft, ready, in_progress, ready_for_review, accepted, merged, deployed, verified, done, blocked, rejected o cancelled. No confundir rama con main, PR con merge, CI con aceptación, merge con despliegue ni despliegue con verificación. Sin autorización explícita no se hace merge, despliegue, cambio de runtime ni acción irreversible. Las reglas recurrentes se incorporan a AGENTS.md; los cambios de estado al roadmap/board; arquitectura a ADR/diagramas; despliegue a runbook, URL, versión y SHA; y trabajos multirrepositorio a un manifiesto coordinador. Nuevo encargo: nueva conversación de Codex. Corrección del mismo encargo: misma conversación y rama. Nuevo resultado importante: chat separado dentro del Proyecto. Tras cambiar AGENTS.md, reiniciar Codex antes del siguiente encargo.

Aplicación

Work prepara y revisa; no sustituye la evidencia de Git, CI o runtime. Codex no debe editar directamente la configuración de Work: Raúl copia este bloque cuando decida adoptarlo.

Sistema de encargos

Cada encargo lógico vive en `docs/encargos/<ID>.md`. El archivo es el contrato durable entre Work, Codex y Git; el chat aporta conversación, no sustituye al contrato.

Flujo mínimo

1. Work prepara el encargo en estado `draft`.
2. Raúl lo aprueba como `ready` y fija rama, SHA base y permisos.
3. Codex lo lee antes de modificar archivos y pasa a `in_progress` cuando corresponde.
4. La rama y el PR reúnen implementación, pruebas y evidencias accesibles.
5. `ready_for_review` permite revisar; no significa `accepted`.
6. Merge, despliegue, verificación y `done` requieren sus gates y evidencias propios.

Estados permitidos: `draft`, `ready`, `in_progress`, `ready_for_review`, `accepted`, `merged`, `deployed`, `verified`, `done`, `blocked`, `rejected` y `cancelled`.

Usa [la plantilla](#). [GOV-001](#) es el primer encargo real versionado.



Estado

draft

Repositorios

- Canónico:
- Relacionados:
- No afectados:

Git

- SHA base:
- Rama:

Objetivo y contexto

Alcance

Fuera de alcance

Restricciones y autorizaciones

- Merge permitido: no
- Despliegue permitido: no

Criterios de aceptación

Pruebas obligatorias

Evidencias accesibles

Impacto documental

Cambio	Documento que debe actualizarse

Riesgos y bloqueos

Rollback

Resultado

Cierre Git

- SHA final:
- PR:
- CI:

Limitaciones y pendientes

Siguiente paso

GOV-001 — Contrato operativo Work–Codex–Git

Estado

`ready_for_review`

Tipo

Gobierno, documentación y proceso. Sin cambios funcionales de producto.

Base y ramas

- Repositorio canónico: `ratienza/infra-replicant-lab`
- Base confirmada de Replicant Lab: `main@92574978ca48ee4a8da48a478f0fc423caffac54`
- Rama de ejecución ya creada: `agent/gov-001-work-codex-git`
- Primera adopción: `ratienza/ErasmusHomes`
- Rama que debe crear Codex en ErasmusHomes desde su `origin/main` real: `agent/gov-001-erasmushomes-adoption`

Este archivo es el contrato canónico del encargo. Codex debe leerlo junto con todos los `AGENTS.md` aplicables antes de actuar.

Objetivo

Institucionalizar el flujo permanente con el que Raúl gestionará proyectos complejos desde varios ordenadores:

- Raúl: Product Owner y autoridad final.
- ChatGPT Work: Project Manager, planificación, contrato, revisión y aceptación.
- Codex: ejecución técnica, pruebas, documentación y evidencias.
- GitHub: fuente de verdad técnica y documental.
- CI: verificación automática.
- Runtime: evidencia de lo realmente desplegado.

El procedimiento debe permitir retomar cualquier proyecto desde otro ordenador o una conversación nueva sin depender del historial de chat.

Después de GOV-001, cada encargo deberá quedar escrito en Git y Codex podrá iniciarlo con una instrucción breve que incluya proyecto e identificador exacto.

Situación que corrige

EH-002R-C ha demostrado varios riesgos que deben quedar prohibidos:

- Confundir rama o PR con `main`.
- Confundir CI verde con aceptación.
- Confundir merge con despliegue.
- Confundir despliegue con verificación.
- Declarar aceptación visual sin preview o capturas accesibles.
- Usar expresiones ambiguas como “ejecuta lo último”.
- Conservar el contrato solamente en un chat.
- Mezclar estado de varios repositorios en una única afirmación.

Alcance en Replicant Lab

1. Nueva sección documental

Crear una sección canónica denominada **Gobierno del trabajo**.

Ruta preferida, adaptable solo si la estructura existente exige otra equivalente:

- `docs/gobierno/flujo-work-codex-git.md`

Añadirla a `mkdocs.yml` de manera visible y coherente, sin alterar ni ocultar la navegación, fichas y secciones existentes.

La página debe ser clara, breve, humana y suficiente por sí sola.

2. Contenido obligatorio

Debe documentar:

1. Roles y responsabilidades.
2. Jerarquía de fuentes de verdad.
3. Jerarquía de contratos:
4. instrucciones del Proyecto de Work;
5. `AGENTS.md` ;
6. `roadmap/estado`;
7. `docs/encargos/<ID>.md` ;
8. `rama/PR`;
9. evidencias;
10. merge, despliegue y verificación.
11. Ciclo completo:
12. planificación;
13. encargo en Git;
14. ejecución;
15. PR y evidencias;
16. revisión;
17. corrección o aceptación;
18. merge;
19. despliegue;
20. verificación;
21. siguiente encargo.
22. Estados permitidos:
23. `draft`
24. `ready`
25. `in_progress`
26. `ready_for_review`
27. `accepted`
28. `merged`
29. `deployed`
30. `verified`
31. `done`
32. `blocked`
33. `rejected`
34. `cancelled`
35. Diferencia exacta entre rama, PR, CI, `main` , runtime y verificación.
36. Rama por encargo lógico; una fase puede contener varios encargos.
37. Reglas para chats:
38. nuevo encargo = nueva conversación de Codex;
39. corrección del mismo encargo = misma conversación y rama;
40. nuevo resultado importante en Work = chat separado dentro del Proyecto;
41. cambio de `AGENTS.md` = reiniciar Codex antes del siguiente encargo.
42. Trabajo desde varios ordenadores:
43. entrar en el repositorio correcto;
44. verificar árbol limpio;

45. actualizar Git;
46. iniciar Codex desde la raíz;
47. indicar el identificador exacto;
48. verificar las instrucciones cargadas.
49. Coordinación multirrepositorio:
50. repositorios afectados y no afectados;
51. rama, PR y SHA por repositorio;
52. dependencias;
53. orden de merge y despliegue;
54. rollback;
55. estado individual.
56. Matriz de impacto documental.
57. Definición de terminado.
58. Checklist para retomar un proyecto después de semanas.
59. Antipatrones prohibidos.

3. Gráficos

Incluir exactamente tres diagramas Mermaid pequeños y legibles:

1. Ciclo Work–Git–Codex.
2. Máquina de estados.
3. Trabajo desde varios dispositivos con GitHub como centro.

Preferir orientación vertical y no superar cinco nodos en horizontal.

4. Instrucciones para Work

Crear un bloque breve y listo para copiar en las instrucciones de un Proyecto de ChatGPT Work.

Ruta preferida:

- `docs/gobierno/instrucciones-proyecto-work.md`

Debe fijar:

- roles;
- Git como fuente de verdad;
- preparación y revisión de encargos;
- estados;
- límites de autorización;
- actualización documental;
- reglas de chats;
- prohibición de confundir rama, `main` y runtime.

Codex no debe intentar modificar directamente la configuración de ChatGPT Work.

5. Sistema de encargos

Crear o adaptar:

- `docs/encargos/README.md`
- `docs/encargos/TEMPLATE.md`

La plantilla debe incluir como mínimo:

- ID, título y estado;
- repositorio canónico y relacionados;
- SHA base y rama;
- objetivo y contexto;
- alcance y fuera de alcance;
- restricciones;
- criterios de aceptación;
- pruebas;

- evidencias;
- impacto documental;
- riesgos y bloqueos;
- permisos de merge y despliegue;
- rollback;
- resultado;
- SHAs y PR finales;
- limitaciones, pendientes y siguiente paso.

Este archivo GOV-001 debe permanecer como primer encargo versionado y ejemplo real.

6. AGENTS.md de Replicant Lab

Actualizarlo de forma breve, sin copiar toda la documentación.

Debe obligar a Codex a:

- leer el encargo identificado;
- trabajar en la rama indicada;
- respetar estados y gates;
- actualizar documentación por impacto;
- publicar evidencias accesibles;
- no hacer merge ni despliegue sin autorización;
- distinguir rama, PR, main, runtime y verificación;
- informar por repositorio en trabajos multirrepositorio.

Primera adopción en ErasmusHomes

Desde el `origin/main` real y limpio de `ratienza/ErasmusHomes`, crear:

- rama `agent/gov-001-erasmushomes-adoption`;
- `AGENTS.md` breve y específico;
- `docs/encargos/README.md`;
- `docs/encargos/TEMPLATE.md`;
- referencia clara al roadmap, board y estado canónicos.

No duplicar toda la página general de Replicant Lab. ErasmusHomes debe contener las reglas operativas suficientes para que Codex pueda trabajar aunque solo tenga abierto ese repositorio.

Matriz documental mínima

Cambio	Actualización obligatoria
Estado de encargo	Roadmap, board o estado canónico
Funcionalidad	README y documentación de usuario
Arquitectura	ADR y diagramas
Despliegue	Runbook, URL, versión y SHA
Varios repositorios	Manifiesto coordinador
Hito aceptado	Changelog y cierre
Regla recurrente	AGENTS.md

“Automático” significa que Codex actualiza únicamente la documentación afectada y CI comprueba la coherencia cuando sea posible. No significa regenerar o modificar indiscriminadamente todos los documentos.

Definición de terminado

GOV-001 estará `ready_for_review` únicamente si:

- existe una sola página canónica;
- aparece en la navegación de Replicant Lab;
- los tres Mermaid renderizan correctamente;
- existe el bloque listo para Work;
- existe el sistema de encargos;
- ambos `AGENTS.md` contienen las reglas operativas;
- ErasmusHomes ha adoptado la estructura;
- build, enlaces y validaciones están verdes;
- existe preview o evidencia visual accesible desde los PR;
- los PR están abiertos y revisables;
- no se ha fusionado ni desplegado nada.

`ready_for_review` no equivale a `accepted`.

Evidencia visual obligatoria

Publicar evidencia accesible desde GitHub, no solamente rutas locales:

- navegación con “Gobierno del trabajo”;
- página completa en escritorio;
- vista móvil;
- los tres diagramas renderizados;
- ausencia de overflow horizontal;
- consola sin errores.

Si no se puede publicar una preview o capturas accesibles, devolver `blocked`; nunca afirmar “listo para aceptación visual” sin evidencia enlazable.

Fuera de alcance y prohibiciones

- No modificar funcionalidades de ErasmusHomes.
- No modificar EH-002R-C ni sus ramas/PR.
- No iniciar EH-003.
- No modificar Apps Launch.
- No modificar Cartera Estratégica, PulaHomes, CV/Firebase, Consumos Cupra ni otros repositorios.
- No tocar datos privados, secretos, tokens, bases de datos o exportaciones.
- No modificar servicios, puertos, contenedores, proxies o runtimes.
- No desplegar en Nexus.
- No hacer merge.
- No editar directamente `main`.
- No reutilizar ramas antiguas.
- No resolver silenciosamente colisiones con trabajo ajeno.

Validaciones

Ejecutar las validaciones indicadas por los repositorios y, al menos:

- build estricto de MkDocs;
- enlaces internos;
- navegación;
- Markdown;
- render real de Mermaid;
- escritorio y móvil;
- overflow horizontal;
- consola;

- coherencia entre plantillas, AGENTS y página canónica;
- árboles limpios tras commit/push;
- CI de ambos PR.

Git y PR

- Continuar Replicant Lab exclusivamente en `agent/gov-001-work-codex-git`.
- Crear la rama de adopción de ErasmusHomes desde su `origin/main` real.
- Commit y push separados por repositorio.
- Abrir PR borrador por repositorio.
- No hacer merge ni despliegue.

Informe final obligatorio

Responder con:

1. Estado: `ready_for_review`, `partial` o `blocked`.
2. Tabla por repositorio:
3. rama;
4. SHA inicial;
5. SHA final;
6. PR;
7. CI;
8. árbol limpio.
9. Archivos creados y modificados.
10. Enlaces a la página, plantillas y bloque de Work.
11. Enlaces directos a las evidencias visuales.
12. Comandos y resultados de validación.
13. Confirmación explícita:
14. no merge;
15. no despliegue;
16. EH-002R-C no modificado;
17. EH-003 no iniciado.
18. Limitaciones reales.
19. Un único siguiente paso propuesto.

Regla de honestidad

No confundir:

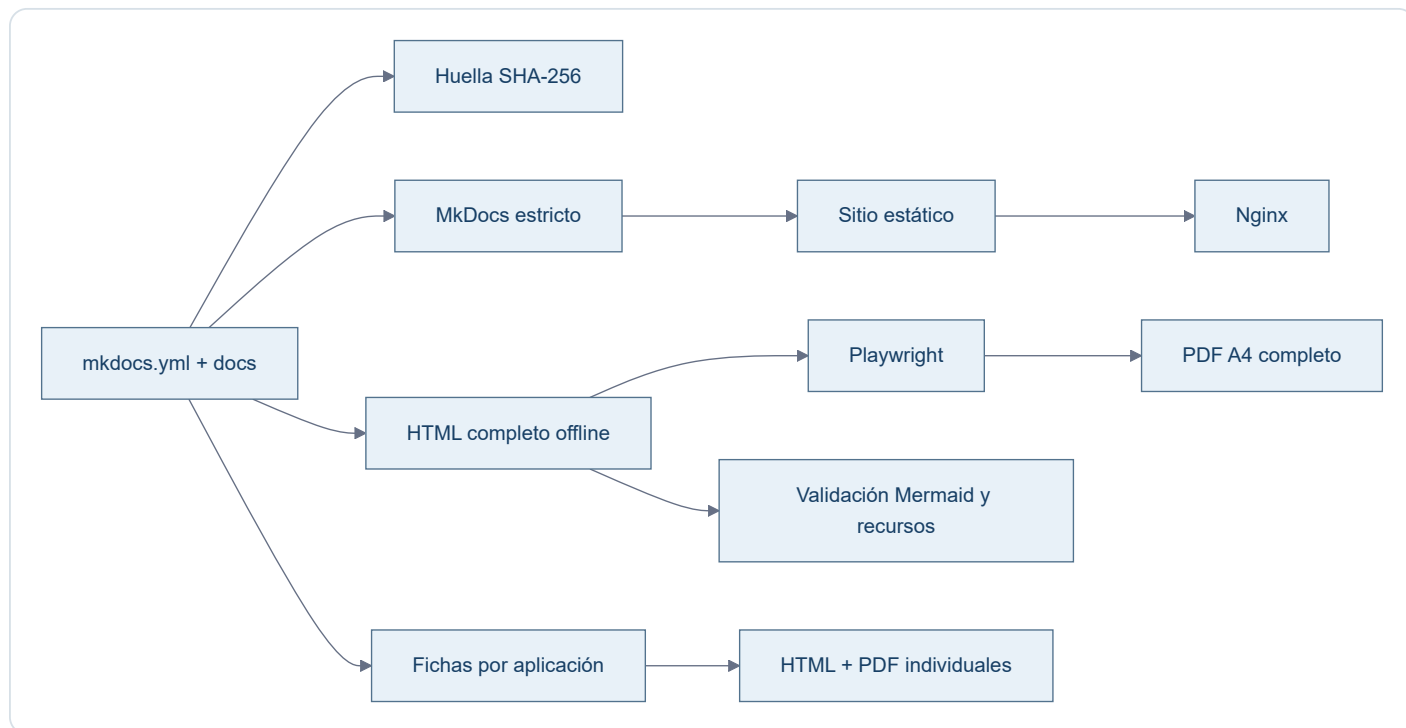
- escrito con validado;
- rama con main;
- PR con merge;
- CI con aceptación;
- merge con despliegue;
- despliegue con verificación;
- captura local con evidencia accesible;
- documentación creada con instrucciones ya pegadas en Work.

Si falta una prueba, debe declararse.

Autodocumentación

Estado implementado

La documentación mantiene una fuente humana canónica (`mkdocs.yml` , `docs/` y recursos) y una cadena reproducible para construir el sitio y sus artefactos derivados.



Herramientas versionadas

- `scripts/docs_pipeline.py` : orquesta construcción, generación, sincronía y validaciones estructurales.
- `scripts/render_portables.mjs` : renderiza Mermaid en Chromium y produce PDF y evidencias.
- `scripts/render_app_portable.mjs` : valida escritorio/móvil y genera cada PDF individual.
- `scripts/portable.css` : presentación compartida por HTML offline y PDF.
- `requirements-docs.txt` : dependencias Python exactas.
- `package.json` y `pnpm-lock.yaml` : Mermaid, Playwright y árbol Node fijados.
- `docs/javascripts/mermaid.min.js` : runtime derivado de la versión Mermaid fijada y comprobado contra `node_modules` .

Huella y reproducibilidad

La huella incluye configuración MkDocs, páginas en `nav` , recursos y herramientas que afectan a los portables. Los HTML deben coincidir byte a byte con la salida esperada. Los PDF se comprueban mediante esa huella visible, cobertura textual y equivalencia de paginación con una regeneración de control.

Temporales

Todos los resultados intermedios viven bajo `.build/docs-pipeline/` . Entornos virtuales, `node_modules` , navegadores, cachés, capturas y reportes no entran en Git.

Flujo de actualización

1. modificar exclusivamente fuentes MkDocs;
2. ejecutar `python scripts/docs_pipeline.py generate ;`
3. revisar el sitio, HTML, PDF y capturas;
4. ejecutar `python scripts/docs_pipeline.py check ;`
5. revisar el diff completo;
6. commit, push y Pull Request;
7. tras merge, reconstruir el servicio estático en Nexus mediante Compose;
8. validar publicación y descargas en Nexus como un estado separado.

Decisiones arquitectónicas

Registro corto de decisiones que no conviene redescubrir.

Decisión	Motivo
Nexus usa IP fija <code>.220</code>	Identidad estable del servidor local
Replicant usa <code>.200</code>	Identidad estable del host físico
Alias locales sin servidor DNS	Replicant resuelve <code>nexus → 192.168.18.220</code> y <code>replicant → 192.168.18.200</code> mediante <code>hosts</code> ; no se añade infraestructura DNS
Docker como estándar	Mantener limpio el host Ubuntu
GitHub como source of truth	Reproducibilidad e histórico
Datos y secretos fuera de Git	Seguridad y separación de responsabilidades
UFW restringe SSH a LAN	Reducir superficie de exposición
No usar Portainer/Webmin/Cockpit por defecto	Evitar complejidad innecesaria
Ramas por cambio lógico	Facilitar revisión y mantener <code>main</code> estable
MkDocs es la fuente documental canónica	<code>mkdocs.yml</code> , <code>docs/</code> y sus recursos definen contenido, estructura y presentación
HTML/PDF son artefactos derivados	Deben sincronizarse mediante un proceso reproducible; nunca prevalecen sobre MkDocs
Nexus sirve una imagen estática Nginx	<code>Dockerfile</code> , <code>Compose</code> y <code>CI</code> comparten build MkDocs estricto; el despliegue requiere reconstrucción
Validación por entorno	Git, prueba local, Nexus y producción son estados distintos y no se extrapolan
Dependencias documentales fijadas	Python/Node, MkDocs, Mermaid y Playwright se resuelven desde lockfiles versionados
Huella SHA-256 para portables	Evita timestamps variables y permite demostrar sincronía inequívoca con las fuentes
PDF tratado como binario	<code>.gitattributes</code> anula <code>diff=astextplain</code> y conversiones CRLF de Git para Windows
Rutas portables únicas	HTML y PDF viven exclusivamente en <code>docs/downloads/</code>
App Launch usa puerto <code>80</code> en Nexus	Permite <code>http://nexus/</code> sin puerto explícito; <code>8080</code> queda libre
App Launch selecciona catálogo al desplegar	Una presentación común y un único <code>apps.json</code> activo por host evitan filtrar enlaces internos
Checkouts no equivalen a servicios	CV y Control de Red pueden existir en disco de Nexus sin declararse desplegados
Fichas técnicas derivadas por aplicación	Cada aplicación tiene HTML/PDF generado desde su Markdown canónico
Salones AV liga <code>8081</code> a la IP LAN	<code>192.168.18.220:8081:80</code> evita publicar el servicio en todas las interfaces y queda versionado desde <code>8c0bc08</code>
App Launch es navegación, no runtime	Una tarjeta puede apuntar a servicios locales o remotos sin trasladar su ejecución al host del launchpad
Consumos Cupra usa Cloud Run	La cadena canónica es GitHub → Cloud Build → Artifact Registry → Cloud Run; DigitalOcean y Nexus no son su producción
CV usa Firebase Hosting	La producción observada pertenece al proyecto <code>replicant-lab</code> ; el Cloud Run placeholder no es producción

Change Log

Acceso estable al histórico diario de Raul Lab.

- [21 de agosto de 2026](#)
- [13 de agosto de 2026](#)
- [9 de agosto de 2026](#)
- [8 de agosto de 2026](#)

Las entradas se ordenan de la fecha más reciente a la más antigua.

Change Log · 21 de agosto de 2026

Auditoría y documentación de PULA

- Clonado `ratienza/Pula` en Nexus como checkout de consulta, sin convertirlo en runtime.
- Auditado `main` (`ff8165c`) como POC privada de búsqueda de apartamentos SCPU en Pula, Croacia.
- Incorporados arquitectura, manual funcional, API REST, riesgos y pendientes propios de PULA.
- Compilados TypeScript y el build Vite/esbuild sobre una copia temporal; PULA permaneció limpio.
- Registrada la ausencia de tests automatizados y de trazabilidad reproducible de Cloud Run.

Corrección de separación de proyectos

- Excluidos de PULA el concepto ErasmusHomes, el ejemplo Bolonia/UNIBO, Smart Deposit Audit y el dossier ReportLab.
- Confirmado que su ubicación canónica es el repositorio independiente `ratienza/ErasmusHomes` .
- Clasificados la etiqueta y los artefactos ErasmusHomes del historial de PULA como contaminación histórica, no como versión o capacidad de PULA.
- ErasmusHomes no fue auditado ni registrado todavía como aplicación funcional.

Documentación integral y portable externo de PULA

- Completadas la especificación de API, persistencia, seguridad y operación reproducible a partir del código de `main` .
- Añadidos el diagrama de arquitectura y el aviso visible que separa PULA de ErasmusHomes.
- Incorporados los pendientes reales de PULA al backlog del laboratorio sin presentarlos como funciones validadas.
- Regenerados desde MkDocs los portables HTML/PDF de PULA y el dossier global mediante el pipeline reproducible.

No se modificaron PULA, ErasmusHomes, Cloud Run, Firebase, Gmail, Gemini, SCPU, DigitalOcean, datos vivos ni credenciales.

Publicación de PULA en App Launch

- Añadida PULA a los catálogos público y Nexus de `ratienza/Apps_Lauch` mediante el squash `2d265ee` .
- Desplegados exclusivamente ambos launchers desde la fuente versionada y verificada su coincidencia exacta con cada `apps.json` .
- Validada con HTTP `200` la URL pública `https://pula-erasmus-housing-automator.ai.studio/` y su destino idéntico desde ambos catálogos.
- Nexus mantuvo estable el contenedor `app-launch` ; DigitalOcean mantuvo Nginx activo.
- Las tarjetas son enlaces: PULA no se trasladó ni se ejecuta en Nexus o DigitalOcean.

No se modificaron PULA, ErasmusHomes, el runtime público de Google, Firebase, Gmail, Gemini, SCPU, datos vivos, secretos ni otros servicios.

Change Log · 13 de agosto de 2026

Auditoría global y App Launch

- Confirmada la implementación multientorno de App Launch en `ced6e14`.
- Validados DigitalOcean por HTTP/HTTPS y Nexus por el puerto `80`.
- Confirmado `8080` libre y `8081-8083` sanos.
- Verificada la separación exacta de catálogos y la ausencia de referencias Nexus en DigitalOcean.

Inventario de aplicaciones

- Auditados repositorio, SHA, arquitectura, despliegue y estado real de ocho aplicaciones.
- Incorporadas fichas individuales para App Launch, Consumos Cupra, CV y Control de Red.
- Ampliadas las fichas de Salones AV, Reserva-Pistas-UTP y Cartera Estratégica.
- Incorporada una ficha individual de Replicant Lab, separada del dossier global.
- Diferenciados explícitamente servicios, checkouts de solo lectura, producción cloud y MVP local.

Evidencias y pendientes

- Reservas: 21 pruebas correctas.
- Consumos: TypeScript y build correctos.
- CV: build y PDF correctos.
- Salones: enlaces locales y rutas publicadas correctos.
- Control de Red: sintaxis PowerShell correcta sin escaneo.
- Cartera: 369 pruebas recogidas; ejecución parcial sin fallos hasta el límite temporal, no declarada como suite completa.
- Conservadas como pendientes la deriva Compose de Salones, la trazabilidad de despliegue de Consumos, la configuración Cloud del CV, los datos versionados de Control de Red y la política global de backups.

No se modificaron CV/Firebase/Cloud Run, Cartera, red, firewall, DNS, certificados, secretos ni datos privados.

Fase 2A · Remediación de Salones AV

- Resuelta la deriva de `compose.yml` mediante el PR `salones-av-valencia-palace#2`.
- Versionado el bind LAN `192.168.18.220:8081:80` en `main` (`8c0bc08`).
- Reconciliado Nexus preservando y comparando el cambio local antes del avance rápido.
- Verificados Compose, blob Git, checkout limpio, bind efectivo y las siete rutas con respuesta `200`.
- No se modificaron contenido AV, fuentes documentales, datos, otros contenedores ni producción DigitalOcean.

Consolidación final pre-Cartera

- Cerradas documentalmente las fases 2A, 2B y 2C sin reabrir remediaciones.
- Consumos Cupra queda trazado desde `9f66a368` hasta Cloud Run `cosnumos-cupra-00009-pon`; DigitalOcean y Nexus no son su runtime.
- El CV queda identificado en Firebase Hosting del proyecto `replicant-lab`, operativo en `cv.raulatienza.com`, con deuda aceptada POST-CARTERA.
- App Launch queda definido como catálogo y navegación: una tarjeta no demuestra runtime local.
- La navegación histórica se reduce a hitos y las pruebas detalladas permanecen en las fichas de las ocho aplicaciones.

9 de agosto de 2026

Change Log diario de Raul Lab. Reúne los cambios documentales, técnicos y operativos de la fecha, del hito más reciente al más antiguo.

Cierre integral de Reserva-Pistas-UTP · validación final 10/08/2026

- Los [PR #4](#), [#5](#) y [#6](#) integraron mediante squash el motor de sincronización, la corrección del usuario de runtime y la guía de despliegue validada. El borrador #1 quedó cerrado al ser sustituido.
- La aplicación dejó de exponer contraseñas y de incluir credenciales en las tareas. El almacenamiento incorpora validación estricta, IDs estables, bloqueo entre procesos, escritura atómica, copias, restauración, ledger y detección de conflictos.
- El panel permite vista previa y confirmación explícita en ambas direcciones, evita dobles ejecuciones y presenta conflictos solo con campos no sensibles. La transferencia se realiza desde Nexus mediante clave dedicada y comando SSH forzado en DigitalOcean.
- Se superaron 21 pruebas, compilación Python, sintaxis JavaScript, configuración y construcción Docker, controles HTTP y revisión de logs.
- Antes de intervenir se conservaron copias de seguridad; los datos funcionales y los ficheros locales de credenciales y notificaciones de producción mantuvieron su integridad.
- Se replicaron de DigitalOcean a Nexus 18 registros —12 cancelados y 6 reservados—, sin tareas activas, duplicados ni conflictos. Después, ambas vistas previas convergieron en 18 elementos sin cambios y el documento normalizado compartió el hash `db9a806429479d9e83c83724ca67b5e072ca2ab29b94fbde9dbd19475342dc09`.
- Nexus quedó operativo en `192.168.18.220:8083` y DigitalOcean en `https://app.raulatienza.com/padel/`; no se alteraron DNS, certificados, puertos ni la topología pública. El [issue #2](#) quedó cerrado con evidencias.

Corrección final de experiencia documental · PR #11

- El [PR #11](#) integra la corrección final de cronología, visión transversal de repositorios y experiencia portable.
- La sección **Cambios** pasa a presentarse como **Change Log**, con una cronología única, fechas visibles y contexto técnico y operativo.
- **Pendientes** amplía su alcance a los ocho repositorios vinculados a Raul Lab comprobados en GitHub; `ratienza/python` queda excluido expresamente.
- El HTML independiente recupera una experiencia multipágina dentro de un único fichero autocontenido: índice persistente, una vista documental activa, anterior/siguiente, fragmentos directos, historial atrás/adelante, búsqueda local y adaptación móvil.
- El PDF mantiene su modelo editorial completo; HTML y PDF continúan generándose desde la fuente MkDocs canónica y no desde exportaciones antiguas.

18:55 · Estado real de Nexus registrado · PR #10

- El [PR #10](#) integró la evidencia de despliegue y cerró los estados que todavía figuraban como pendientes.
- `main` quedó en `cd09dec4cb083f7a761e09648e23404cfd35da0c`.
- Nexus quedó sincronizado, con Nginx estático estable en `192.168.18.220:8082`, cinco Mermaid y descargas verificadas.

18:39 · Pipeline reproducible y runtime estático · PR #9

- El [PR #9](#) integró el pipeline reproducible de documentación.
- MkDocs —`mkdocs.yml`, `docs/` y sus recursos— quedó reafirmado como fuente canónica de contenido, estructura y presentación.
- Docker, Compose y CI convergieron en `mkdocs build --strict` y una imagen final Nginx; desaparecieron `mkdocs serve`, los `bind mounts` y el runtime de desarrollo en Nexus.
- Mermaid 11.16.1 pasó a servirse localmente, sin CDN, y los cinco diagramas quedaron disponibles también offline.
- Se generaron un HTML autocontenido completo y un PDF A4 completo desde las 27 páginas de navegación.
- Las rutas canónicas quedaron fijadas en `docs/downloads/Replicant-Lab.html` y `docs/downloads/Replicant-Lab.pdf`; se eliminó el HTML duplicado obsoleto.
- La validación local incluyó MkDocs estricto, enlaces, recursos, Docker, escritorio, móvil, HTML `file://`, PDF y ausencia de CDN, localhost, WebSocket y recarga en vivo.
- El squash resultante fue `f457c57b472515afe46025078c7342dd5a75a7f1`.

Despliegue y validación real en Nexus

- Se actualizó exclusivamente `/opt/apps/infra-replicant-lab` mediante avance rápido y `docker compose up -d --build`.
- El contenedor `infra-replicant-docs` quedó activo con Nginx `1.27.4`, reinicios `0` y publicación `192.168.18.220:8082 → 80`.
- Se comprobaron navegación, recursos, presentación de escritorio y móvil, cinco Mermaid, HTML offline y PDF completo.
- Las descargas reales coincidieron byte a byte con Git:
- HTML: `17a2746d2e11e26d0302a57633c5b1b05119348d8e4a4629f00d67ab8cfce6a1`.
- PDF: `028a135d7645912569c5d6c6c13b7c885b250084e3b269add7d10f89f627e13b`.
- Los logs finales no mostraron errores, `404`, recarga en vivo, WebSocket ni dependencias esenciales externas.

17:11 · Reconciliación y consolidación documental · PR #8

- El [PR #8](#) reconcilió las fuentes MkDocs con el estado comprobado de GitHub y Nexus.
- Se separaron con claridad Replicant/Hyper-V, Nexus, DigitalOcean y los repositorios propios de cada aplicación.
- README, navegación, operación, despliegue, aplicaciones y pendientes quedaron alineados; el procedimiento de Nexus distinguió primer arranque de actualización ordinaria.
- El squash resultante fue `9c31728bfb0a4399dc736bea59f7ed50ed480bf3`.

16:07 · Contrato operativo raíz · PR #7

- El [PR #7](#) incorporó `AGENTS.md`.
- El contrato fijó la jerarquía entre verdad técnica versionada, documentación MkDocs canónica y artefactos HTML/PDF derivados.
- También formalizó separación de entornos, protección de datos, flujo rama–pruebas–PR–merge y prohibición de desplegar o modificar producción sin autorización.
- El squash resultante fue `01d859fac7b7907ae92cf65bda2ad50cfb0e738f`.

8 de agosto de 2026

Change Log diario de Raul Lab. Se conserva íntegramente el histórico previamente registrado para esta fecha.

- Replicant queda con IP fija `192.168.18.200` .
- Nexus queda con IP fija `192.168.18.220` mediante Netplan.
- SSH por clave consolidado.
- UFW activado con SSH permitido solo desde la LAN.
- Se verifican Docker, Git y actualizaciones automáticas.
- Salones AV queda accesible desde la LAN en `192.168.18.220:8081` .
- Replicant Lab queda publicado en Nexus por `192.168.18.220:8082` .
- Reserva-Pistas-UTP pasa de un staging manual a un despliegue Docker Compose completo en Nexus: backend Python + Nginx en red privada Docker, acceso LAN por `192.168.18.220:8083` .
- El backend de Reserva-Pistas se hace configurable mediante `APP_HOST` , `APP_PORT` y `APP_DATA_DIR` , manteniendo valores por defecto compatibles con el despliegue anterior.
- La persistencia de Reserva-Pistas se separa en `/opt/data/reserva-pistas` y se monta como `/app/data` .
- El backend Docker se ejecuta como UID/GID `1000:1000` y ambos servicios usan `restart: unless-stopped` .
- Se valida HTTP `200` , persistencia tras recreación de contenedores y recuperación automática tras reinicio completo de Nexus.
- UFW permite `8083/tcp` exclusivamente desde `192.168.18.0/24` .
- Se formaliza la configuración Nexus en `ratienza/Reserva-Pistas-UTP` y se integra en `main` mediante PR #3.
- DigitalOcean permanece sin cambios y continúa como producción de Reserva-Pistas-UTP.
- Queda pendiente sincronizar a Nexus el histórico/estado vigente de `tasks.local.json` desde DigitalOcean, verificando antes que no existan tareas activas que puedan duplicarse.
- La portada documental incorpora accesos de descarga al HTML autónomo y al PDF de Replicant Lab.
- DNS local y backups se mantienen pendientes conscientemente.
- Se consolida `infra-replicant-lab` como documentación viva.